

THE UNIVERSITY OF QUEENSLAND
AUSTRALIA

EVALUATING ROBUST RETRIEVAL PIPELINES

by
OLIVER CARNEW
School of Information Technology and Electrical Engineering,
University of Queensland.

Submitted for the degree of

Bachelor of Engineering (Pass/Honours)

in the division of Electrical and Biomedical

NOVEMBER 2023

November 5, 2023

The Dean
School of Engineering
University of Queensland
St Lucia, Q 4072

Dear Professor Michael Bruenig,

In accordance with the requirements of the degree of Bachelor of Engineering (Honours) in the division of Computer Systems Engineering/Electrical and Electronic Engineering, I present the following thesis entitled ”EVALUATING ROBUST RETRIEVAL PIPELINES”.

This work was performed under the supervision of Prof. Guido Zuccon. I declare that the work submitted in this thesis is my own, except as acknowledged in the text and footnotes, and has not been previously submitted for a degree at the University of Queensland or any other institution.

Yours sincerely,

OLIVER CARNEW

To . . .

Acknowledgments

The author would like to acknowledge the contributions of Professor Guido Zuccon for his assistance and direction, instrumental in the formation of this work.

Abstract

Pretrained language models (PLMs) are effectively utilised across the gamut of information retrieval (IR) tasks. The effectiveness of a neural ranking model is tested within IR benchmarks which instantiate a single query for the information need, and the ranking of documents of passages is subsequently evaluated. Recent research shows that human generated user queries have significant variation across the same information need and adversarial queries significantly degrade the efficacy of IR systems. This project aims to evaluate the robustness of recent PLM IR pipelines on large datasets with novel and established query variation techniques, to establish how variations which maintain query semantics affect systems' effectiveness. The query variation generators constructed and utilised within the study show that evaluation of traditional, neural and pretrained language model based retrieval systems lose retrieval effectivity of ~45% . Robust information retrieval pipelines are instrumental to successful practical use by users.

Contents

List of Figures 5
List of Figures 6
1 Abstract 7
2.2 Related Work 8
2.2.1 Query Variation 8
2.2.2 Model Explorations 9
3 Background and Literature Review 11
Introduction 11
3.1 A Brief History of Information Retrieval 12
3.1.1 Introduction 12
3.1.2 Historical Context 12
3.1.3 Transition to Machine Learning 13
3.1.4 The Rise of Neural Networks 13
3.1.5 Influence of Pretrained Language Models (PLMs) 14
3.1.6 A Typical Modern Retrieval Pipeline 14
3.2 Ranking Architectures 15
3.2.1 Lexical and Traditional IR Models 16
3.2.1.1 BM25 17
Merits and Constraints 18
Merits 19
Weaknesses 19
3.2.2 Relevance Model 3 19
Merits 20
Weaknesses 20
3.2.3 Neural Information Retrieval (Neural IR) 21
3.2.3.1 Introduction to Neural Information Retrieval (IR) 21

3.2.3.2 KNRM 21
Merits 22
Weaknesses 22
3.2.3.3 Conv-KNRM 23
Merits 23
Weaknesses 24
3.2.4 Pre-trained Language Models 24
3.2.4.1 Bidirectional Encoder Representations from Transformers (BERT) 25
Merits 28
Weaknesses 28
3.2.4.2 Expansion via Prediction of Importance with Contextualization (EPIC) 28
Merits 30
Weaknesses 30
3.2.4.3 Text-to-Text Transfer Transformer (T5) 30
Core Mechanics of T5: 31
Merits 32
Weaknesses 32
3.1 Query Variations 33
3.1.1 Query Variations Database 33
3.2 Datasets 34
3.2.1 MS MARCO 34
3.2.2 ANTIQUE 34
3.2.3 Dataset Wrangling 35
3.3 OpenAI GPT Model 35
3.3.2 Merits 38
3.3.3 Weaknesses 38

4 Methodology 40
4.1 Query Variation 40
4.1.1 LLM Query Variation 40
4.1.2 Character Deletion 42
4.1.3 Lemmatization Query Variation 42
4.2 Ranking Models 43
4.2 Evaluation 44
5 Results 45
5.1 Query Generation 45
5.1.1 GPT Query Generation 45
5.1.2 Character Deletion Query Variation 47
5.1.3 Lemmatisation Query Variation 47
6 Discussion 51
7 Conclusions and Future Work 54
Future Work 54
Bibliography 57

List of Figures

Figure 1; Simple representation of typical architecture of an ad-hoc retrieval system. Image Source: Jurafsky and Martin 2009 pg.782 [55] 13
Figure 2; Multistage architecture of modern information retrieval system. Image Source: J Guo et al., 2022 [65] 14
Figure 3; Full formulation of the canonical BM25 ranking formula, as shown directly in The Fourth Text REtrieval Conference by S.E Robertson et al. 1994 [73] 18
Figure 4; Excluding the output strata, identical architectural frameworks are employed throughout both pre-training and fine-tuning stages. Pre-trained model parameters are uniformly adopted as initial conditions across diverse downstream applications. Subsequent fine-tuning is applied to all parameters. The '[CLS]' token denotes a unique initiator symbol preceding each input instance, while '[SEP]' functions as a distinctive delimiter token, for instance, bifurcating questions and answers. From “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding” (pg. 3) by Devlin et al. 2018 [17] 26
Figure 5; BERT employs a bidirectional Transformer, conditioning on both left and right contexts across all layers. GPT uses a left-to-right Transformer, predicting each token based on its predecessors. ELMo utilizes bidirectional LSTMs, generating features from both directions independently. BERT and GPT are fine-tuned, adapting pre-trained models to specific tasks, whereas ELMo provides features for task-specific model enhancement. From “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding” (Appendix A, pg. 13) by Devlin et al. 2018 [17] 27
Figure 6; Sample of 'ir_datasets' catalog page, showing a subsection of the datasets available from MSMARCO for easy use [109] 35
Figure 7; The Transformer overall architecture using stacked self-attention and pointwise, fully connected layers for both the encoder and decoder, shown in the left and right halves of the layout. From “Attention Is All You Need” by Vaswani et al. 2017 [62] 37
Figure 8; Process flow for generating a query variation using OpenAI GPT API 41
Figure 9; Example of increased specificity and query length in returned query variation 41
Figure 10; Demonstration of random character deletion on an original query (from TREC-DL-2019) 42
Figure 11; Demonstration of lemmatisation on an original query from ANTIQUE 43
Figure 12; Increase in query variation length as a function of original query length (reported as a percentage). Shorter queries are more often represented by longer generated queries. For ANTIQUE dataset 45
Figure 13; Increase in query variation length as a function of original query length (reported as a percentage). Shorter queries are more often represented by longer generated queries. For TREC-DL-2019 dataset 45
Figure 14; Increase in query variation length as a function of original query length (reported as a percentage). Perturbed queries are all either equal or shorter in length to original queries. For TREC-DL-2019 dataset 47
Figure 15; Perturbed queries are all either equal or shorter in length to original queries. For ANTIQUE dataset 48

List of Figures

TABLE 1; COMPARISON BETWEEN DIFFERENT QUERY SETS AND THEIR LENGTHS AFTER VARIATION 46
TABLE 2; COMPARATIVE EFFICACY (NDCG@10) ACROSS VARIED RETRIEVAL METHODOLOGIES IN TREC-DL-2019 AND ANTIQUE DATASETS SUBJECT TO QUERY MODIFICATIONS. EMBOLDENED FIGURES REPRESENT PEAK VALUES FOR RESPECTIVE MODELS, BLUE TEXT INDICATES HIGHEST PEAK VALUES FOR QUERY VARIATION EVALUATIONS. CELL COLOURS ARE USED TO DELINEATE ORIGINAL QUERY EVALUATION VS PERTURBATED QUERIES (GREEN = ORIGINAL, BLUE = PERTURBED). ALL QUERY VARIATION METHODS RESULT IN STATISTICALLY SIGNIFICANT ($P<0.05$) EFFECTIVITY DROPS ACROSS BOTH DATASETS (ANTIQUE AND TREC-DL-2019. 50

1 Abstract

Pretrained language models (PLMs) are effectively utilised across the gamut of information retrieval (IR) tasks. The effectiveness of a neural ranking model is tested within IR benchmarks which instantiate a single query for the information need, and the ranking of documents of passages is subsequently evaluated.

Recent research shows that human generated user queries have significant variation across the same information need and adversarial queries significantly degrade the efficacy of IR systems. This project aims to evaluate the robustness of recent PLM IR pipelines on large datasets with novel and established query variation techniques, to establish how variations which maintain query semantics affect systems' effectiveness.

2.2 Related Work

To put the proposed research effort into context, prior research in query variation and neural IR model research analysis is described.

2.2.1 Query Variation

Several previous studies have suggested that evaluation of IR systems should account for multiple instantiations of the same information need, i.e., query variations, because these variations can affect the performance of ranking models [1-4].

Zuccon et al. [4] proposed a mean-variance framework for comparing different IR systems that explicitly accounts for query variations. Bailey et al. [1, 2, 5] argued that an effective model should be consistent to different query variations and proposed a measure of consistency to complement effectiveness measurements.

Moreover, query variations can be employed to improve the overall effectiveness of ranking models. For example, some studies have combined different rankings obtained from query variations [6-8], while others have modelled the relevance of multiple query variations [9]. Query variations have also been shown to be helpful for the problem of query performance prediction [10].

In addition, previous studies have proposed various methods for automatically generating query variations [11]. These include query expansion through a relevance model, generating variations based on a query-url click graph [9], or using an external knowledge base as in the work of Chakraborty et al. [12].

Penha et al. [13] developed a notably different exploration of query variation in several ways. They note that their methods do not require access to external corpora, a relevance model, or a query-url click graph. Second, their focus is not solely on generating queries that improve effectiveness but also on generating queries that are likely to occur in practice. Finally, their approach involves generating query variations according to a taxonomy that allows for diagnosing the effectiveness of ranking models based on the types of variations that are most detrimental [13].

The authors' work builds on that of MacAvaney et al. [14] that proposes a framework for assessing the sensitivities of neural IR models to various perturbations to documents in the dataset, extending a selection of user created perturbations to the query set.

Penha et al. devise several methods of generating user inspired query variations. Care was given to maintaining semantic and pragmatic meanings in the query text while altering query syntax. The chosen perturbations were fielded from the UQV100 [2] data set of user generated queries to a given information need. The taxonomy developed by the authors identified 4 appropriate query transitions, termed 'misspelling, naturality, ordering and paraphrasing'.

Generative methods for generating relevant query variations for each transition were utilised for the test query sets in the ANTIQUE [15] and TREC-DL-2019 [16] datasets. The resulting 2040 (10 methods iterated over 204) queries were used to evaluate the effectiveness of a number of traditional/lexical (BM25, RM3), neural (KNRM, CKNRM) and transformer neural IR (BERT, EPIC, T5) models, resulting in an ~20% decrease in performance [13].

However, the query transitions identified by the authors are not exhaustive and recent advancements in large language models (LLMs) releases prompt further investigation. Motivated by these limitations, this proposal aims to evaluate the robustness of recent IR models on established and novel post-transition query sets to better inform the construction of robust IR pipelines that are more comprehensively informed by user variation.

2.2.2 Model Explorations

Pre-trained transformer-based language models such as BERT [17], T5 [18] predicated a paradigm shift in information retrieval (IR) due to their success on several notable IR benchmarks. The gains in effectiveness for IR and natural language processing (NLP) tasks which utilise BERT and adjacent models lead to focused research on understanding and defining the behaviour and reasoning behind the models' success [14, 19-21].

The work of Camara and Hauff [20] as well as that of MacAvaney [14] focuses on expanding and developing understanding of neural IR model axioms, developing axiomatic diagnostic datasets to complement the work of Rennings et al [22]. Three key strategies of assessing neural IR model behaviour are identified. These are measure and match, NLP dataset transfer and dataset document manipulation. Additionally, Wu et al. [23] investigate neural ranker robustness through three criteria; *"1) The performance variance under the independent and identically*

distributed (I.I.D.)setting; 2) The out-of-distribution (OOD) generalizability; and 3) The defensive ability against adversarial operations" (Lu et al. 2022, pg. 1).
These efforts differ from the proposed focus on altering user-generated queries in that the aim is to replicate and synthesise the user variance aspect of query generation, in a way that is inspired by practical user cases. This proposal aims to illustrate the robustness of neural IR models more comprehensively to transformations that naturally occur, and how they manifest in the relative effectiveness of contemporary models.

3 Background and Literature Review

Introduction

The effectiveness of pre-trained transformers, such as BERT (and variants), T5 and ELECTRA, in Information Retrieval (IR) tasks has been demonstrated extensively [24-28]. While IR benchmarks evaluate the performance of ranking models based on a single query, user-generated queries for a fixed information need are highly variable and affect retrieval efficacy [1, 2, 5]. Additionally, ranking models are highly vulnerable to inaccuracies when tested with adversarial examples [29, 30]. Several studies have shown that ranking evaluations in IR benchmarks (such as in TREC campaigns) should consider multiple user query variations, given their impact on ranking ability [2-4, 31, 32]. Several studies have also shown that IR pipelines which utilise query variation and rank fusion achieve more effective ranking systems [6, 8, 13, 33, 34].

Penha, G et. al [13] have investigated how robust IR pipelines are with generative query variations. Their research created a taxonomy based on the manual annotation of transformations in UQV100 and generated query variations using different automatic methods for each category. Four types of syntactic query transformations (misspelling, naturality, ordering and paraphrasing) were identified from the query datasets. The study re-ranked the top 100 documents of two datasets (ANTIQUES and TREC-DL-2019) and found that retrieval pipelines are not robust to query variations maintaining the same content, with a drop of ~20% in effectiveness [13].

Development of larger datasets with a greater number of test queries [35], query generation techniques [36-38], more effective PLM IR pipelines for evaluation campaigns (such as TREC-2021-DL) [35] as well as more comprehensive evaluation metrics and test sets [4], especially with focus on query variability and system robustness [2, 34], necessitate further investigation.

Additionally, further attention must also be given to generating novel query variations that affect syntax through methods of typography errors, lemmatization, and extraneous string insertion. These query variations may be tested against a different or more modern document corpus. Studying the robustness and response of ranking models when faced with query variability models allows for better understanding of

model effectiveness and suggests methods to improve existing ranking pipeline robustness.

3.1 A Brief History of Information Retrieval

3.1.1 Introduction

Text Information Retrieval (IR) fundamentally revolves around the principle of efficiently identifying and retrieving pertinent textual resources, predominantly documents or passages, that resonate with user queries formulated in natural language.

Stemming from an era inundated with burgeoning data needs, these systems serve as pivotal instruments in combatting information inundation, finding applicability in realms such as:

- question answering systems [39] [40]
- dialog systems [41, 42]
- entity linking [43, 44]
- personal assistants
- chatbots
- digital libraries

and of course, the colossal domain of Web search [45]. As multiple records may be relevant, the documents procured by various retrieval pipelines are often ranked by their relevance score to the user's query. [38]

This field has experienced significant evolution, ranging from early text indexing methods in the 1950s to the recent innovations leveraging pretrained language models (PLMs).

3.1.2 Historical Context

From the 1950s, research has been directed towards text indexing using representative terms. Traditional retrieval systems mainly rely on the matching of textual terms between query and documents [46]. However, term-based retrieval systems suffer several limitations in particular instances of multiple meanings per word (polysemy), synonymy (multiple words per meaning), and lexical gaps between the query and the documents [47]. Researchers identified weaknesses in implementation whereby term-based models:

1) May suffer from the ’vocabulary mismatch problem’ [48, 49] due to the independence assumption
2) May not capture document semantics well by ignoring term ordering information [50]

A significant breakthrough was the vector space model, based on the 'bag-of-words' premise [51]. ‘Bag-of-words’ approaches are so named because they ignore syntactic information. It conceptualized both queries and documents as sparse term-based vectors. Lexical similarity, supported by the inverted index data structure [52], was the primary metric for relevance. Retrieval mechanisms subsequently incorporated probabilistic frameworks like the BM25 model and utilized statistical language modelling methods for text ranking [53, 54]. These early retrieval methods, based upon heuristic strategies or simplified probabilistic principles, lay the foundation of modern information retrieval systems.

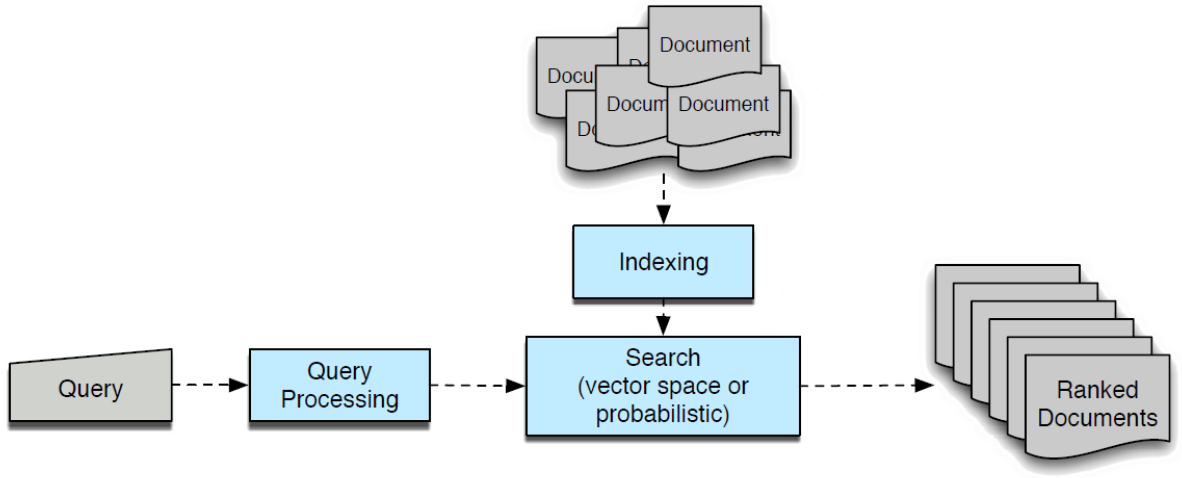


Figure 1; Simple representation of typical architecture of an ad-hoc retrieval system. Image Source: Jurafsky and Martin 2009 pg.782 [55]

3.1.3 Transition to Machine Learning

With the advent of machine learning, the concept of learning to rank emerged [56]. Here, ranking functions were informed by hand-crafted features and subsequently trained using relevance judgements (binary or annotated). While they provided flexibility, a significant reliance remained on human-driven feature engineering [57].

3.1.4 The Rise of Neural Networks

The renaissance of neural networks heralded the ability to autonomously derive text features, eliminating manual interventions. Deep learning methods [58] enabled the

extraction of query and document representations from labelled data [59]. These representations, termed as dense vectors or embeddings, pivoted towards capturing semantic nuances unlike the traditional sparse vectors [60]. This shift heralded the Neural Information Retrieval (Neural IR) paradigm with earlier methods preceding the introduction of PLMs termed as pre-BERT models [19, 61].

3.1.5 Influence of Pretrained Language Models (PLMs)

PLMs, driven by the Transformer architecture [62], have monumentally [19] uplifted the capabilities of text retrieval systems. By encoding a rich tapestry of semantic knowledge, they excel in text representation. Their commendable performance in natural language processing [17] catalysed their assimilation into text retrieval, spawning the next-gen PLM-based dense retrieval models. These models have consistently overshadowed their predecessors in benchmark datasets [35, 63]. Their dominance is rooted in their adeptness at comprehending intricate queries and the accessibility of expansive labelled training datasets, such as MS MARCO [16], MSMARCOv2 [16] and Natural Questions [64]. Large, labelled datasets coupled with advancements in computing power have greatly impacted the field of Natural Language Processing (NLP), enabling researchers to use deep learning techniques for a variety of innovations. The techniques developed have been used to improve traditional retrieval systems and overcome term-based retrieval system limitations.

3.1.6 A Typical Modern Retrieval Pipeline

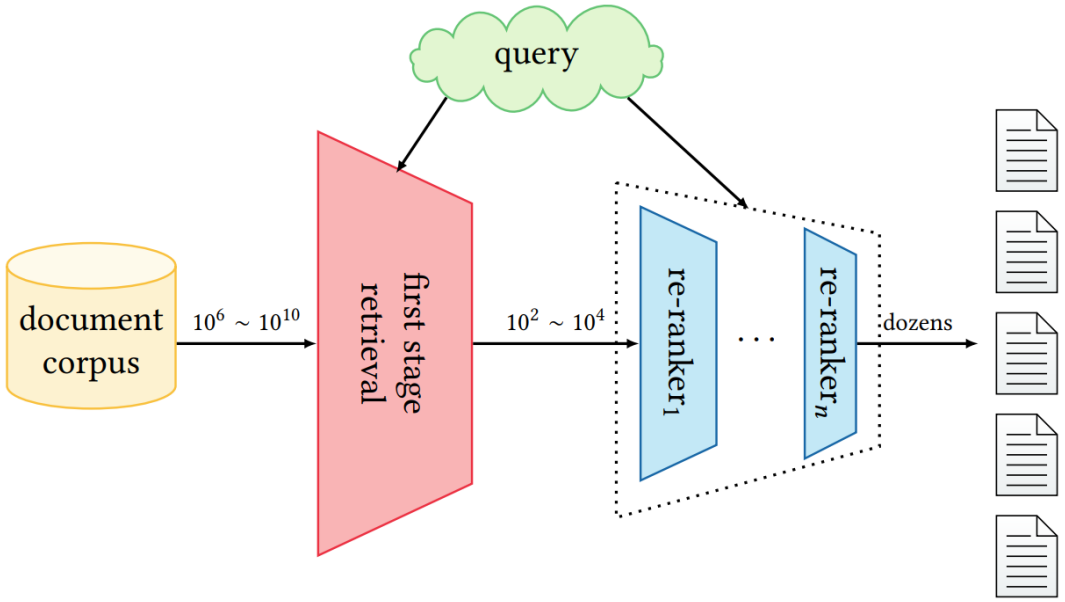


Figure 2; Multistage architecture of modern information retrieval system. Image Source: J Guo et al., 2022 [65]

To optimise both search efficiency and efficacy, contemporary search systems predominantly utilise a multi-stage ranking pipeline, as illustrated in Figure 2. The preliminary retrieval phase is designed to source an initial array of candidate documents from an expansive repository, leveraging economical ranking models bolstered by purpose-built indexing frameworks.

Subsequent phases involve intricate, high-performance ranking models to refine and enhance the list of documents ranked by the prior phase. This sequential "retrieval and re-ranking" methodology has garnered acceptance in both academic [66, 67] and commercial spheres [68], setting new standards on several IR benchmarks [69, 70].

Beyond the pipeline's structural design, a pivotal component to efficacious retrieval lies in the system's nuanced comprehension of both the query and document. This facilitates the surfacing of content germane to the user's informational requisites. Consequently, throughout the pipeline, the incorporation of semantic models is paramount, albeit with distinct objectives and prerequisites at each juncture.

In the initial retrieval phase, the objective is a comprehensive identification of relevant documents from the entire collection, necessitating semantic models that can swiftly and efficiently ensure maximal recall. In contrast, during subsequent re-ranking stages, the scope narrows to a limited set of documents, enabling the deployment of advanced semantic architectures with an emphasis on precision, strategically placing the most pertinent documents to the top of the list. Historically, there has been an observable evolution in the re-ranking phases, transitioning from rudimentary probabilistic models, through learning-to-rank models, and culminating in contemporary neural ranking methodologies The advent of BERT-esque pre-training tasks applied to cross-attention models has facilitated enhanced contextual representation and deeper symbiosis between query-document dyads, marking a significant enhancement in re-ranking efficacy [21, 24, 37, 71]. However, the computational demands of these models often render them ill-suited to manage the high volume of queries each paired with an extensive set of candidate documents characteristic of the initial retrieval stage.

3.2 Ranking Architectures

The following aims to highlight the ranking models and enhancements which will comprise the 7 ranking approaches tested within this research document (note that

these approaches re-rank the top 100 documents returned by an initial BM25 retrieval for each query). The ranking approaches which will be considered are:

- Lexical
  - BM25 [53, 72, 73]
  - RM3 [74]
- Neural
  - KNRM [75]
  - CKNRM [75]
- Transformer
  - BERT [24]
  - T5 [26]
  - EPIC [76]

3.2.1 Lexical and Traditional IR Models

This section delves into the fundamental aspects of classical term-based techniques utilised in primary-stage retrieval and ranking within this project. Central to the discussion are the vector space model (VSM), probabilistic retrieval frameworks, and relevance modelling approaches for IR. Collectively, these techniques construct representations for queries and documents grounded in the bag-of-words (BoW) hypothesis. Within this paradigm, texts are depicted as a collection of individual words, with lexical structure and sequence notably overlooked. The representation functions, designated ϕ and ψ, predominantly rely on manually curated features like word frequency. The dimensions of such representations, notably $k1$ and $k2$, correspond typically to the size of the vocabulary. While the representation functions for queries and documents diverge, they maintain the hallmark of sparsity, facilitating the application of the inverted index for streamlined retrieval1.

Historically, the vector space model (VSM) is a pioneering term-based technique [46]. Within the VSM, queries and documents are conceptualised as expansive sparse vectors situated within a shared vector space. Here, every dimension aligns with a vocabulary term. The significance of each dimension may be calibrated via various mechanisms, including term frequency (TF), inverse document frequency (IDF), or a synthesis of both [77, 78]. The relevance between a query and a document can be ascertained through the similarity measure, frequently employing cosine similarity. The resulting scores subsequently inform the prioritisation of pertinent documents for a query. The VSM has subsequently laid the foundation for subsequent IR methodologies. Both the probabilistic retrieval model and the language model for IR can be conceptualised as derivations of the VSM, albeit with distinctive weighting systems.

Probabilistic techniques stand as one of the earliest structured models within IR, embedding probability theory to ascertain the likelihood of a document's relevance to a specific query.

3.2.1.1 BM25

Introduced in 1994 by Robertson and Walker [79] BM25 is the most successful member of the BM (Best Match) family. BM25 builds on the original BM retrieval formula which itself expands on the earliest TF*IDF constructions [80]. The BM25 foundational concepts derive from Harter's 2-Poisson model, originally formulated for document indexing efficacy [81] and its integration with the canonical probabilistic framework for relevance, as delineated by Robertson and Sparck Jones [82].

TF*IDF aims to approximate a relevance score for a text match, by considering:

TF: Term Frequency – how often does a term appear in a document

IDF: Inverse Document Frequency – Inverse measure of how often a document contains a given term, e.g. how rare a term is in a broader corpus. Used to weed out common words that relay no broader relevance (e.g. and, the, a).

BM25 improves upon TF*IDF by accounting for term-saturation and document length. TF: Term frequency is adjusted to include a tuneable value 'k' such that an increase in Term Frequency contributes to relevance score less and less. Typically $k = 1.2$.

$$T.F = (T.F)/(T.F + k)$$

Equation 1; Term frequency value incorporating tuneable 'k' value

IDF: Contribution of IDF is attenuated for documents which are shorter than the average document length within the corpus. This is achieved by multiplying the tuneable 'k' value from earlier by:

$$1 + b + b \times \frac{Document\ Length}{Average\ Document\ Length}$$

Equation 2; Implementation of tuneable document length equation

Where the parameter 'b' is used to tune the effect of deviation from average document length. Typically, $b = 0.75$.

Thus the BM25 ranking formula is as shown in Figure 3:

$$\sum_{T \in Q} w^{(1)} \frac{(k_1+1)tf}{K+tf} \frac{(k_3+1)qtf}{k_3+qtf} + k_2.|Q|.\frac{avdl-dl}{avdl+dl} \quad (1)$$

where

Q is a query, containing terms T

$w^{(1)}$ is the Robertson–Sparck Jones weight [3]

$$\log \frac{(r+0.5)/(R-r+0.5)}{(n-r+0.5)/(N-n-R+r+0.5)}$$

of T in Q

N is the number of items (documents) in the collection

n is the number of documents containing the term

R is the number of documents known to be relevant to a specific topic

r is the number of relevant documents containing the term

K is $k_1((1-b)+b.dl/avdl)$

k_1, b, k_2 and k_3 are parameters which depend on the database and possibly on the nature of the topics. For the TREC–4 experiments, typical values of k_1, k_3 and b were 1.0–2.0, 8 and 0.6–0.75 resp., and k_2 was zero throughout.

tf is the frequency of occurrence of the term within a specific document

qtf is the frequency of the term within the topic from which Q was derived

dl is the document length (arbitrary units)

$avdl$ is the average document length.

Figure 3; Full formulation of the canonical BM25 ranking formula, as shown directly in The Fourth Text REtrieval Conference by S.E Robertson et al. 1994 [73]

Merits and Constraints

BM25 is often considered a useful benchmark for ranking pipeline performance, whilst exhibiting notable merits and constraints.

Merits

BM25 is a pre-eminent ranking function, acclaimed for its simplicity and its adeptness at curating relevant search outcomes. It is a mainstay in the realm of information retrieval due to these attributes [53].
It proficiently integrates term frequency with document length normalisation, effectively rectifying the bias induced by document size disparities.
With its robust capacity to manage voluminous document assemblages, BM25 stands as a scalable solution, tailor-made for real-world search applications.

Weaknesses

A significant limitation lies in BM25's omission of semantic and contextual comprehension within queries and documents, which may impair the ranking precision for certain queries [83].
The algorithm's underpinning assumption is the statistical independence of query terms — a supposition which falters in the face of term co-dependencies.
BM25's emphasis on term frequency and document length comes at the expense of other critical elements, such as the structural intricacies of documents and the incorporation of relevance feedback.

3.2.2 Relevance Model 3

Relevance Model 3 (RM3) is an enhancement of the Relevance Model (RM), which itself extends the query likelihood model by incorporating feedback documents. RM3 particularly stands out as an effective query expansion method within the language modelling approach to information retrieval (IR). It builds upon the initial query by incorporating terms from feedback documents, presumed to be relevant, to enrich the query representation and enhance retrieval performance [84].
The architecture of RM3 involves a combination of the original query and terms selected from a set of top-retrieved documents (pseudo-feedback documents). Specifically, RM3 utilizes a weighted mixture of the original query language model and a language model built from feedback documents. The weights assigned to the original query and the feedback model are crucial, as they balance the influence of the user's original intent against the context gleaned from relevant documents [85, 86].

In terms of efficacy, RM3 has been shown to significantly improve retrieval performance in various settings and datasets. It excels in disambiguating user queries and retrieving documents that might not be retrieved through keyword matching alone due to vocabulary mismatches [74, 85, 87].

Merits and Constraints

The practical effectiveness of RM3 has been assessed across various standard test collections and IR tasks, such as those from the Text Retrieval Conference (TREC) [74]. Merits and weaknesses of RM3 implementation follow below.

Merits

Enhanced Query Representation: By expanding the query with additional relevant terms, RM3 can capture the context better, leading to more accurate retrieval.

Vocabulary Mismatch Mitigation: RM3 addresses the issue of vocabulary mismatch between the query and the documents by incorporating terms from relevant documents.

Robustness: The method has proven effective across various domains and types of information retrieval tasks.

Adaptability: RM3 can be adapted to different IR scenarios, including ad-hoc retrieval and web search.

[74, 85, 86, 88]

Weaknesses

Dependence on Quality Feedback: The efficacy of RM3 is contingent on the relevance of the feedback documents. Poor feedback can degrade performance.

Parameter Sensitivity: The method requires careful tuning of parameters, including the weights for the original query and feedback model.

Computational Overhead: Query expansion and the processing of feedback documents introduce additional computational costs.

Noise Introduction: The inclusion of non-relevant terms from feedback documents can introduce noise and potentially harm retrieval effectiveness.

[74, 85-87]

3.2.3 Neural Information Retrieval (Neural IR)

3.2.3.1 Introduction to Neural Information Retrieval (IR)

The landscape of text retrieval has been significantly reshaped by advancements in machine learning, notably through the application of learning to rank techniques. These methods have historically relied on supervised learning paradigms, utilizing a wide array of features—both lexical and beyond—crafted by human experts to train ranking functions against relevance judgments [56, 57, 89]. While this approach boasts considerable flexibility, it is not without its dependencies on extensive human input for feature engineering.

The resurgence of neural networks has catalysed a pivotal shift in the creation of text retrieval systems, liberating the process from the confines of hand-crafted feature dependency. Deep learning techniques in information retrieval [58] offer an automatic approach to learning representations of queries and documents. By converting text into low-dimensional embeddings within a latent space, these methods allow for relevance to be inferred based on the semantic proximity of the dense vectors—a stark contrast to the classic vector space model's reliance on sparse vectors.

These advancements have ushered in the era of Neural Information Retrieval (Neural IR) [59, 61] which distances itself from traditional retrieval paradigms by not directly mapping embeddings to specific term dimensions. Instead, it seeks to encapsulate latent semantic nuances to facilitate matching.

3.2.3.2 KNRM

Within the sphere of Neural IR, the Kernelized Neural Ranking Model (KNRM) stands out as a significant contribution. KNRM combines the representational prowess of neural networks with the interpretability and structure of kernel pooling to effectively capture the nuances of semantic matching between queries and documents [75].

The architecture of KNRM utilizes a dual embedding strategy, where both queries and documents are transformed into dense vector representations. These embeddings are then subjected to a series of kernelized pooling operations, which enable the model to encapsulate varying degrees of similarity and match different n-gram patterns across the query and document texts. The kernels essentially act as

soft histograms, capturing the count of embedding similarities at different levels, allowing for a nuanced interaction feature space that can be fed into a learning-to-rank framework [75, 90].

Efficacy in information retrieval pipelines is evident as KNRM leverages the deep semantic understanding inherent to neural embeddings, providing a significant improvement over traditional lexical matching approaches [60]. The model's end-to-end learning capability allows it to adaptively tune both the text representations and the matching function to the task at hand, resulting in higher retrieval performance on benchmark datasets [91].

Merits

Automatic Feature Learning: KNRM eliminates the need for manual feature engineering, reducing human effort and expertise required in the retrieval process [61].

Semantic Matching: It captures deep semantic relations beyond exact lexical matches, providing a more sophisticated understanding of text [91].

End-to-End Learning: The model is trained in an end-to-end manner, optimizing both representation and ranking simultaneously [75].

Interpretability: Kernel pooling offers a form of interpretability by mapping neural representations to similarity scores that can be understood and analysed [75].

Weaknesses

Computational Intensity: The model's complexity and the need for learning dense representations can lead to higher computational costs [92].

Data Dependency: KNRM relies on large amounts of labelled training data to learn effective representations and ranking functions [93].

Limited Contextual Understanding: As a pre-BERT model, KNRM might not capture contextual dependencies as effectively as later models that utilize transformers [24, 59].

Static Representations: Unlike contextualized embeddings from transformers, KNRM’s embeddings are static and do not change based on surrounding text, potentially limiting their expressiveness.

In conclusion, KNRM's approach to information retrieval exemplifies the capabilities and potential limitations of neural network applications in text ranking tasks. Despite

its strengths in semantic matching and automatic feature learning, it also presents challenges related to computational demands and the static nature of its representations.

3.2.3.3 Conv-KNRM

The Convolutional Kernel-based Neural Ranking Model (Conv-KNRM) is a sophisticated evolution of the Kernel-based Neural Ranking Model (KNRM), which is designed for information retrieval tasks. The core innovation of Conv-KNRM lies in its ability to capture soft matches of n-grams between query and documents, a crucial enhancement over the original KNRM, which focused primarily on single terms [91, 94].

Architecture: The Conv-KNRM employs a convolutional neural network (CNN) to encode n-grams of various lengths, effectively capturing local contextual information within the text. These encoded n-grams are then matched using a kernel-pooling technique that allows for the matching of semantically similar phrases, even if they are not exact matches. The resulting soft match signals are then combined through multiple kernel pooling layers, which model the relevance of these soft matches at different granularity levels [75, 91].

Efficacy: The efficacy of Conv-KNRM can be attributed to its n-gram matching capability, which provides a more nuanced understanding of text semantics compared to KNRM. This architecture allows for better handling of synonymy and paraphrasing, leading to improvements in retrieval performance across various benchmarks. Conv-KNRM has been shown to outperform KNRM and other baseline models, especially in scenarios where understanding the context and subtle linguistic differences is crucial for effective retrieval [75].

Merits

Contextual Understanding: By using convolutional layers to capture n-gram representations, Conv-KNRM better understands the context around terms.

Soft Matching: The kernel pooling layers are adept at capturing varying degrees of match between query and document n-grams, allowing for effective soft matching [91].

Versatility: The model has shown to perform well across different languages and information retrieval tasks.

Semantic Matching: Conv-KNRM excels at matching based on semantic meaning rather than just keyword overlap [91].

[61, 95, 96]

Weaknesses

Computational Overhead: The convolutional layers and complex architecture lead to increased computational demands compared to simpler models like KNRM.

Data Hungry: To fully leverage its architecture, Conv-KNRM requires substantial amounts of training data to learn effective n-gram representations.

Interpretability: With increased complexity, the model's decision-making process becomes less transparent compared to more traditional models.

Adaptability: While it excels in ad-hoc search, adapting Conv-KNRM to specialized domains may require significant fine-tuning.

[61, 91, 95, 96]

In conclusion, Conv-KNRM builds upon the strengths of KNRM by integrating a convolutional approach to capture the nuances of language better. It effectively pushes the envelope in the field of information retrieval by leveraging the subtleties of semantic understanding inherent in n-grams. However, these advancements come with trade-offs related to computational efficiency and the need for extensive data.

3.2.4 Pre-trained Language Models

The domain of Information Retrieval (IR) has witnessed a transformative impact with the introduction of pre-trained language models (PLMs) such as BERT [17], EPIC [76], and T5 [18]. Unlike static word embeddings, which can only offer a limited representation of words devoid of context, transformers, with their self-attention mechanism, provide a nuanced understanding of language nuances. These mechanisms allow PLMs to generate contextually enriched word embeddings where the representation of each token dynamically responds to its surrounding textual landscape [97].

The architecture of PLMs involves a combination of encoder and decoder models that use stacked transformers. Encoder-only models like BERT and its optimized variants, RoBERTa [98] and DistilBERT [99], focus on understanding the input text

by generating a rich, contextualised embedding for each token. On the other hand, encoder-decoder models like BART [100] and T5 are designed to produce new sequences based on the input text, which is highly relevant for tasks that involve text generation.

In the context of neural IR, pre-trained language models have been especially effective. They enable a nuanced understanding of queries and documents, thereby greatly enhancing the relevance of the results returned by IR systems. These models are fine-tuned for IR tasks to calculate a relevance score between a query and a document, which is critical for accurately ranking the documents.

When comparing PLMs to traditional neural IR models, several contrasts emerge. Neural IR models often relied on handcrafted features or more primitive forms of word representations, which limited their ability to capture the intricacies of language. PLMs, with their sophisticated architecture, can inherently understand complex language patterns and dependencies, which is a significant step forward for IR systems.

The efficacy of PLMs in IR is largely attributed to their pre-training on extensive corpora, which equips them with a broad understanding of language. Subsequent fine-tuning for specific IR tasks allows these models to transfer that general understanding to specialized domains, a process requiring comparatively lesser computational resources and time.

3.2.4.1 Bidirectional Encoder Representations from Transformers (BERT)

BERT introduced a new paradigm for the utilization of pre-trained language representations. Its architecture, as described by Devlin et al. in 2018, leverages a multi-layer bidirectional Transformer encoder, based on the foundational work by Vaswani et al. (2017), to create deeply contextual language models [17, 62].

The core mechanics of BERT revolve around two distinct training strategies: pre-training and fine-tuning. During pre-training, BERT is exposed to a vast corpus of unlabelled text, allowing it to learn language patterns and representations across a variety of pre-training tasks. One of these tasks is the Masked LM (MLM), where 15% of input tokens are randomly masked and the model learns to predict these

missing pieces, thereby gaining a deeper understanding of context and language structure [17].

Additionally, BERT incorporates a Next Sentence Prediction (NSP) task, which involves predicting whether a pair of sentences naturally follow each other. This task enhances BERT's ability to understand sentence relationships, a critical aspect for many downstream tasks such as Question Answering (QA) and Natural Language Inference (NLI) [17].

For fine-tuning, BERT's parameters, already informed by the broad context of the language from its pre-training, are further refined on labelled data from specific downstream tasks. This phase involves minimal task-specific architecture changes, relying instead on the universality of the pre-trained model, which is adjusted to the nuances of the particular task at hand.

Figure 4 is an image taken from the seminal works of Devlin et al. giving a broader understanding of the training tasks and structures utilised by implementations of BERT

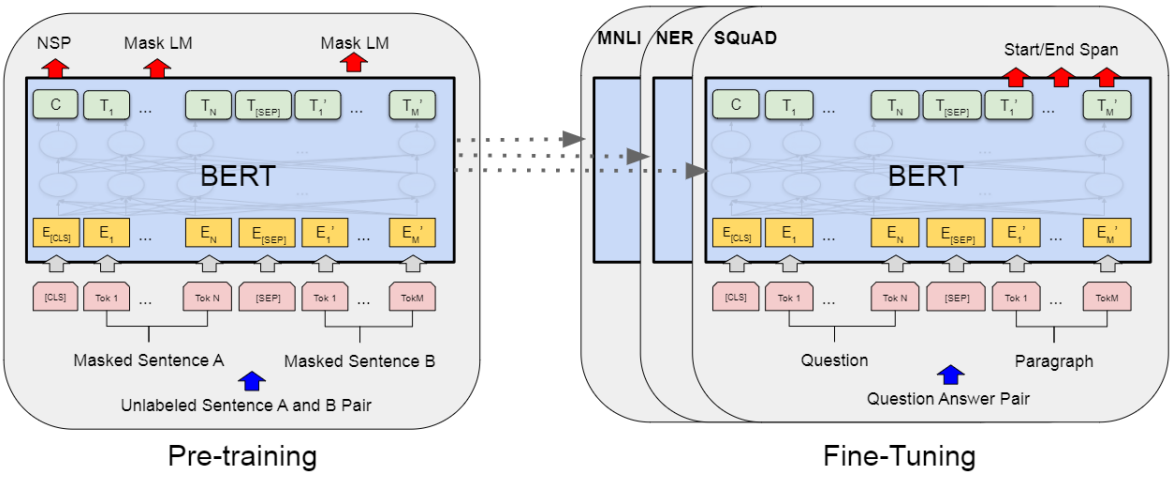


Figure 4; Excluding the output strata, identical architectural frameworks are employed throughout both pre-training and fine-tuning stages. Pre-trained model parameters are uniformly adopted as initial conditions across diverse downstream applications. Subsequent fine-tuning is applied to all parameters. The '[CLS]' token denotes a unique initiator symbol preceding each input instance, while '[SEP]' functions as a distinctive delimiter token, for instance, bifurcating questions and answers. From "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" (pg. 3) by Devlin et al. 2018 [17]

In information retrieval (IR) pipelines, BERT's efficacy is underscored by its ability to grasp the subtle semantic relationships within language. When applied to IR, BERT enhances the retrieval process by effectively capturing the intent behind queries and improving the relevance of the retrieved documents. Its bidirectional nature allows it to surpass traditional unidirectional models that fail to fully encapsulate the context of words within queries and documents [101].

The input representation used by BERT is a combination of token embeddings, segment embeddings, and position embeddings. This amalgamation enables BERT to understand not just the individual words, but also the context of a word based on its position and the relationship between sentences within a single input.

BERT's architecture and training mechanisms represent a significant advancement in the field of information retrieval. By transferring its pre-trained parameters to downstream tasks, BERT brings a nuanced understanding of language that traditional IR models, often based on keyword matching, cannot achieve. The depth and bidirectionality of its language comprehension have led to state-of-the-art performance across a range of IR tasks and benchmarks [19, 101].

The efficacy of BERT in IR is evident from its performance on benchmark datasets. After fine-tuning, BERT has been shown to outperform many traditional IR models on tasks such as ad-hoc document retrieval, passage ranking, and question-answering. It achieves this by effectively encoding semantic relationships between queries and documents, thus improving ranking and retrieval results [19, 24].

The differences between different pre-training architectures are shown in Figure 5

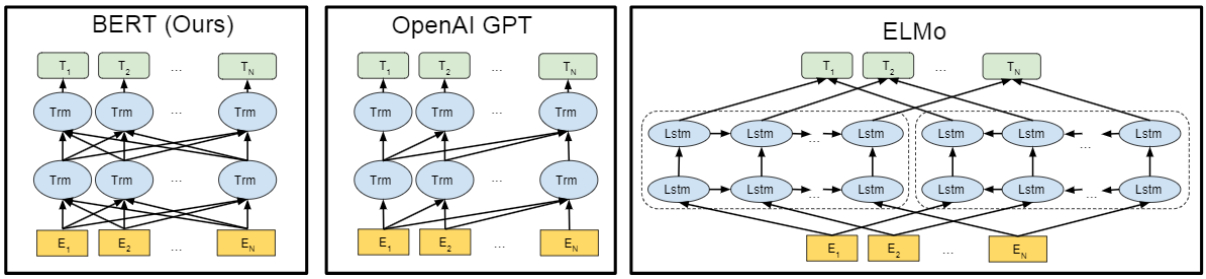


Figure 5; BERT employs a bidirectional Transformer, conditioning on both left and right contexts across all layers. GPT uses a left-to-right Transformer, predicting each token based on its predecessors. ELMo utilizes bidirectional LSTMs, generating features from both directions independently. BERT and GPT are fine-tuned, adapting pre-trained models to specific tasks, whereas ELMo provides features for task-specific model enhancement. From

"BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" (Appendix A, pg. 13) by Devlin et al. 2018 [17]

Merits

Contextualized Embeddings: Captures the context of words more effectively than prior models, leading to a better understanding of query and document semantics.

Fine-tuning Flexibility: Can be adapted to specific IR tasks and domains, improving its applicability and performance.

Transfer Learning: Pre-trained on large corpora, BERT can be fine-tuned with smaller datasets while still achieving high performance, reducing the need for extensive domain-specific data.

State-of-the-Art Performance: Consistently achieves new benchmarks in IR tasks, demonstrating its robustness and accuracy [17].

Weaknesses

Computational Resources: Requires significant computational power for pre-training and fine-tuning, which can be a barrier for some applications.

Fine-Tuning Sensitivity: Performance can be highly sensitive to the fine-tuning process, necessitating careful hyperparameter optimization.

Interpretability: Like many deep learning models, BERT's decision-making process is not easily interpretable, which can be problematic in scenarios requiring explanation.

Overfitting Risk: Due to its complexity, there is a risk of overfitting, especially when fine-tuned on smaller, less diverse datasets.[21]

[17, 19-21, 24, 28, 44, 71, 98, 99, 102]

3.2.4.2 Expansion via Prediction of Importance with Contextualization (EPIC)

EPIC, or Expansion via Prediction of Importance with Contextualization, is a novel passage retrieval framework that enhances the representation-based ranking approach. Unlike conventional retrieval systems that struggle with passage retrieval due to the limited textual context, EPIC introduces three critical advancements [76, 103]:

Importance Modelling: EPIC leverages a contextualized language model to determine the salience of each term within a passage, allowing for a more nuanced understanding of term significance beyond mere frequency counts.

Passage Expansion: By disseminating the computed importance across semantically similar terms, EPIC expands the passage representation, enriching it with contextually relevant information that may not be explicitly present in the original text.

Interpretable Lexicon Grounding: The model ensures that the generated representations are lexicon-grounded, which enhances interpretability. This aspect is crucial for understanding why certain passages are retrieved in response to a query, thus providing insights into the model's decision-making process.

These representations, both for queries and documents, can be pre-computed during the indexing phase to minimize query-time latency, thus offering an efficient re-ranking method. EPIC demonstrates superior performance on the MS-MARCO passage ranking dataset, achieving a notable MRR@10 of 0.304 with an average query latency of just 78ms on standard hardware. Notably, this latency can be reduced further to 68ms with the pruning of document representations, with minimal impact on effectiveness. [16, 76]

The EPIC framework utilizes a representation-centric neural ranking approach, where models are trained to encode queries and documents into a consistent vector space of predetermined dimensionality. The architecture assumes that both queries and documents are constructed from sequences of terms derived from a defined vocabulary set V. Initial representation involves transforming these sequences into vector sequences via a contextualized language model such as BERT [24]. Specifically, a query q is encoded as a sparse vector, while a document is encoded as a dense vector d. Both vectors are passed through a ‘softplus’ function to ensure no negative importance scores for any terms.

The document vectors are additionally passed through a sigmoid function and a max function to further refine the representations of the document. The ranking score is then derived by calculating the similarity between these vectors, employing the dot product as the metric for assessing vector similarity. [76]

Merits

Efficiency: EPIC allows for the pre-computation of document representations, thus significantly reducing the computational load during query retrieval [76, 104].

Effectiveness: It achieves high ranking performance, surpassing previous importance-modelling and document expansion methods [104].

Interpretability: The direct correspondence of representation dimensions to lexicon terms provides clear interpretability of the model's functioning.

Low Latency: EPIC can operate with low latency, which is competitive with other first-stage retrieval methods, making it practical for real-time search applications.[105]

Reduced Storage: The ability to prune document representations contributes to reduced storage requirements without sacrificing retrieval quality.

Weaknesses

Complexity of Language Model: EPIC's reliance on a contextualized language model might introduce complexity that could be prohibitive for extremely large-scale applications or those with strict computational resource constraints.

Potential Overfitting: Given that EPIC fine-tunes the importance of terms, there is a potential risk of overfitting to specific datasets or domains, which may limit its generalizability.

Interpretability vs. Complexity Trade-off: While EPIC is designed to offer interpretability, the intricate nature of its contextualized language model may still pose challenges for users to fully grasp the nuances of its decision-making process.[76]

EPIC's contributions are distinct and its code, along with detailed instructions and trained models, have been made publicly accessible for reproducibility through OpenNIR.

3.2.4.3 Text-to-Text Transfer Transformer (T5)

The T5 (Text-To-Text Transfer Transformer) architecture introduced by Raffel et al.[18] is a seminal development in the field of Natural Language Processing (NLP) that addresses the heterogeneity of model architectures and training objectives across different tasks. The architecture builds upon the transformer model

introduced by Vaswani et al. [62] employing its encoder-decoder structure as a universal interface for a wide range of NLP tasks [18].

In their seminal work, Nogueira et al. (2020) put forward a novel application of generative pre-trained transformer models (PTMs), specifically the Text-to-Text Transfer Transformer (T5), to the task of relevance modeling within information retrieval systems. The T5 model, which operates on a universal text-to-text framework, was configured to perform the ranking task through a specialized input template. This template, structured as "Query: [q] Document: [d] Relevant:", frames the ranking problem as a binary classification task, with the model outputting a discrete label "true" or "false" to signify document relevance relative to the query. To quantify the relevance score of a document, the probability associated with the "true" token, as produced by the model, is employed. This probabilistic output is subjected to a softmax normalization across the logits of both "true" and "false" tokens. The softmax function ensures that the scores are comparative and reflect a probability distribution over the possible relevance classifications, thereby facilitating a robust ranking of documents based on their computed relevance to the query [106].

Core Mechanics of T5:

The model used in T5 is roughly equivalent to the original Transformer architecture proposed by Vaswani et al. [62] apart from removing the Layer Norm bias, placing the layer normalization outside the residual path, and using a different position embedding scheme. [18].

Input/Output Representation (Text-to-Text Framework): T5 standardizes NLP tasks into a unified text-to-text format. This format involves prepending a task-specific prefix to the input text that cues the model on the expected task, allowing for a seamless switch between tasks like summarization, translation, and classification using the same model parameters and loss function.

Span Corruption Objective: Unlike BERT [17], which masks individual tokens, T5 employs a span-based masking strategy where contiguous sequences of tokens are replaced with a sentinel token. This approach, akin to a span-level denoising autoencoder, challenges the model to reconstruct the missing text spans from context, aiding in learning a deeper contextual representation.

C4 Dataset: The training of T5 involves a vast corpus of clean English text, the 'Colossal Clean Crawled Corpus' (C4) dataset. By processing this extensive corpus, T5 benefits from a broad and rich linguistic input, which enhances its ability to generalize across different tasks. Easily available through the TensorFlow Datasets library.

Scaled Models: T5 comes in various sizes, from the smaller model with 60 million parameters to the largest one with 11 billion, allowing researchers and practitioners to choose a model size that fits their computational budget and task requirements.[18]

Merits

Flexibility: The unified approach to NLP tasks reduces the complexity involved in task-specific data processing and model tuning.

Transfer Learning Capability: The pre-training phase on the C4 dataset equips T5 with a broad understanding of language, which can then be fine-tuned on task-specific datasets to achieve state-of-the-art results.

Scalability: The various sizes of T5 models provide options for both high-performance computing environments and more constrained settings.

Open Source: By releasing T5 and the C4 dataset openly, Google has facilitated a wave of research and application development in NLP.

Weaknesses

Computational Requirements: Training and fine-tuning T5, especially its larger variants, demand significant computational resources, which can be a barrier for smaller entities.

English Centric: The C4 dataset is predominantly English, which may limit T5's effectiveness in other languages.

Potential for Bias: Given that C4 is sourced from the web, T5 may inherit and propagate biases present in the training data.

Complexity for Deployment: The size and complexity of T5 models can make them challenging to deploy in production, especially for real-time applications.

3.1 Query Variations

IR benchmarks organized as part of TREC or other evaluation campaigns, evaluate the effectiveness of ranking models for small sets of topics and their corresponding relevance judgments [16]. Importantly, each topic is typically represented by a single query. However, previous research has shown that queries created by users given a fixed information need may vary widely [4, 31].

The Query Track within the Text Retrieval Conference (TREC) investigated the effect of multiple user-originated queries, each formulated to fulfill an identical information requirement, on the evaluation of retrieval effectiveness. The analysis established that there is significant variability within topics and the queries associated with a single topic, while the variability of systems was found to be less pronounced [3]. A particular issue that emerges is the extent to which current corpora represent a comprehensive range of user queries, particularly when there is a broad spectrum of queries pertaining to a solitary topic [107].

Moffat et al. [108] scrutinized the sufficiency of relevance assessments in a conventional single-query dataset, given the presence of such query fluctuations. Their findings indicate that a substantial quantity of the documents returned by these diverse queries lacked judgments, notably those ranking high in the retrieval order. Therefore, there exists a need to develop retrieval pipelines which are robust and able to integrate query variation as an inherent factor, thereby confronting the complexities identified in preceding research.

3.1.1 Query Variations Database

In the UQV100 [2] database, crowd workers on average created 58 unique queries for a given information need as instantiated as a backstory. The following backstory is a sample given by the authors of UQV100 [2]:

> “Having heard of the pristine environment in Maryland, you have long dreamed of taking a fishing holiday there. However, you think that you may need a fishing license in Maryland. How do you get one?”

UQV100 was created using the ClueWeb12-B corpus as the underlying set of documents. The additional resources for the test collection consist of: 100 backstories and their paired TREC topic/subtopic id; corresponding normalized and spell-corrected query variations (an average of 58 per backstory) and effort estimates.

3.2 Datasets

Dense retrieval models necessitate extensive, annotated datasets to fine-tune pre-trained language models (PLMs), which marks a divergence from traditional retrieval systems. Recent years have seen the public release of various information retrieval datasets, complete with relevance assessments, which have markedly propelled the field of dense retrieval forward. These datasets are systematically classified into three primary categories based on their inherent tasks: information retrieval, question answering, and miscellaneous tasks.

3.2.1 MS MARCO

MAchine Reading COmprehension dataset designated as MS MARCO [16]. This extensive dataset encapsulates 1,010,916 anonymized queries, originally sourced from Bing's search query logs, and each is accompanied by a human-generated response. Additionally, it includes 182,669 answers that have been entirely rewritten by humans. Supplementing these queries, the dataset integrates 8,841,823 text passages, which were derived from 3,563,535 unique web documents obtained via Bing searches. These passages are instrumental in assembling the natural language responses. Within the MS MARCO dataset, certain queries may yield multiple or no corresponding answers.

The dataset is engineered to support three distinct tasks, each escalating in complexity: (i) to ascertain the answerability of a query within a given context passage set and to derive and synthesize an answer analogous to human reasoning, (ii) to construct a coherent answer based on the pertinent passages when provided with the context of both the question and passages, and (iii) to sequence the passages retrieved in response to a query.

What distinguishes MS MARCO from existing publicly available datasets in the domains of machine reading comprehension and question-answering is its vast scale and the employment of authentic user-generated search inquiries. The real-world basis and magnitude of the dataset are posited as advantageous for the evaluation and benchmarking of models designed for machine reading comprehension and question-answering tasks. [16]

3.2.2 ANTIQUE

The ANTIQUE dataset comprises a compendium of 2,626 open-domain, non-factoid queries spanning a broad spectrum of categories. This collection features 34,011

manually curated relevance annotations. The queries originated from actual users of a community-driven question-and-answer platform, specifically Yahoo! Answers. To establish the relevance of each response to the corresponding query, a crowdsourced approach was utilized for the annotation process.

3.2.3 Dataset Wrangling

The administration of data pertinent to Information Retrieval (IR) research presents a multifaceted challenge. Comprehensive dataset documentation is often dispersed across various online platforms, and upon acquisition of the dataset, researchers must navigate a plethora of diverse data structures. Even elementary data configurations may exhibit intricate variances unique to each dataset, necessitating careful consideration for effective application.

In response to these complexities, MacAvaney et al. present "ir_datasets," a robust yet streamlined utility designed to streamline the acquisition, management, and execution of common operations on datasets employed in IR studies [109]. The utility's primary emphasis is on textual datasets relevant to ad-hoc search inquiries. It offers access to a vast array of IR datasets and benchmarks via both a Python API and a command-line interface. This utility represents the most comprehensive of its genre currently available. Its integration with prevalent IR indexing and experimental frameworks underscores its practical value in the domain.

Both ANTIQUE and MSMARCO can be interfaced with directly through the ‘ir_datasets’ tool. Figure 6 shows part of the listing for msmarco for the passage ranking task datasets.

Dataset	docs	queries	qrels	scoreddocs	docpairs	qlogs
msmarco-passage	✓					
msmarco-passage/dev	↑	✓	✓			
msmarco-passage/dev/2	↑	✓	✓			
msmarco-passage/dev/judged	↑	✓	↑			
msmarco-passage/dev/small	↑	✓	✓	✓		

Figure 6; Sample of 'ir_datasets' catalog page, showing a subsection of the datasets available from MSMARCO for easy use [109]

3.3 OpenAI GPT Model

GPT-4, an advanced iteration of the Generative Pre-trained Transformer models, manifests an evolution in the architecture and capabilities of large language models

(LLMs). Built upon the transformer architecture, it exemplifies significant enhancements over its predecessors, like GPT-3, in terms of size, complexity, and fine-tuning methods. The model architecture is designed around a deep neural network with a greater number of parameters, which enables a more intricate understanding and generation of natural language [110].

The model is constructed using a stack of transformer blocks, each consisting of multi-head self-attention mechanisms paired with fully connected feed-forward neural networks. This configuration is adept at handling sequential data and extracting patterns across different positions within the input text [62]. The increased parameter count (albeit undisclosed by OpenAI) offers an amplified comprehension of syntax and semantics, enhancing the model's ability to generate human-like text and understand nuanced language.

Layer normalization is another critical feature of GPT's architecture. It standardizes the inputs across the features for each layer, as described by Ba, Kiros, and Hinton [111]. By doing so, it stabilizes the learning process and allows for faster training of very deep networks.

The self-attention mechanism is pivotal to the model's design. It allows each position in the encoder to attend to all positions in the previous layer of the encoder — essentially, each word or token looks at every other word in the sentence when predicting the next word. The multi-head aspect of this mechanism enables the model to focus on different positions and semantic features simultaneously, a sort of parallel processing of linguistic information Position-wise feed-forward networks are applied to the output of the attention layer and are composed of two linear transformations with a ReLU activation in between. These networks contribute to the model's ability to represent positional information and to capture the sequential nature of language. The now ubiquitous transformer model architecture presented by Vaswani et al. is shown in Figure 7, depicting the series of connected layers, with

particular significance placed on the attention layers.

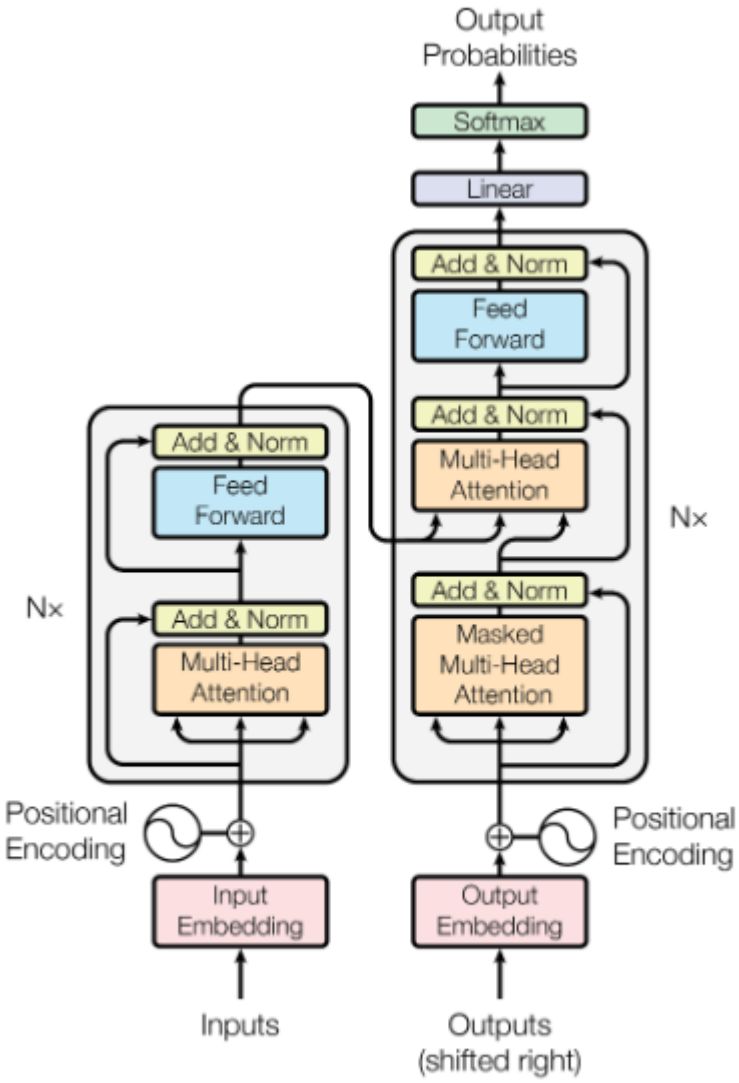


Figure 1: The Transformer - model architecture.

Figure 7; The Transformer overall architecture using stacked self-attention and pointwise, fully connected layers for both the encoder and decoder, shown in the left and right halves of the layout. From "Attention Is All You Need" by Vaswani et al. 2017 [62]

Training such models requires a vast and diverse dataset. As highlighted in research by Radford et al.[112], the construction of a GPT model involves unsupervised learning from a corpus of text that encompasses a wide array of topics. This allows the model to develop a broad understanding of language and context. However, the exact dataset composition and the training methodologies employed remain proprietary to OpenAI and are not fully disclosed in the literature.

The efficacy of GPT-4 in natural language comprehension and generation is underpinned by its sophisticated unsupervised learning regime. By ingesting a colossal dataset drawn from diverse textual sources, GPT-4 learns to predict the next word in a sequence, refining this ability over countless iterations to produce remarkably cogent and context-aware text outputs.

In the realm of information retrieval, GPT-4 exhibits proficiency in understanding queries, identifying relevant information, and generating coherent, concise, and relevant responses. It employs an improved version of machine learning techniques such as few-shot and zero-shot learning, which allows it to perform tasks with minimal instruction or examples [113].

Comparatively, GPT-4 distinguishes itself from other contemporary models, such as BERT or its direct predecessor, GPT-3, in terms of scale and the resulting nuanced understanding of language. While BERT excels in understanding word context in bidirectional ways, GPT-4's transformer model allows for more flexible and generative capabilities, making it superior in tasks involving complex language generation.

3.3.2 Merits

Advanced text generation and completion capabilities with high coherence over long passages.

Improved contextual understanding allows for more nuanced conversation and information retrieval.

Enhanced few-shot learning enables effective task performance with limited examples.

Scalability, allowing the integration with various applications and platforms for versatile usage scenarios.

API Access allows for easy and recursive calls, including conversational and successive contextual prompts, though calls are rate limited and are not free

3.3.3 Weaknesses

Although reduced, there remains a propensity for **generating plausible but incorrect** or nonsensical information.

The model can still **exhibit biases** present in the training data, despite efforts to mitigate them.

The vast amount of **computational resources** required for training and operation may limit accessibility and increase environmental impact.

Dependency on the quality and variety of training data can lead to gaps in knowledge or understanding, especially for information emerging post-training.

4 Methodology

The project requires variation of queries found within TREC-DL-2019 and ANTIQUE, and then subsequent evaluation using described ranking architectures. Resulting rankings are then compared.

4.1 Query Variation

We consider the original queries of two datasets: TREC-DL-2019 and ANTIQUE. The queries considered

Dataset	**TREC-DL-2019**	**ANTIQUE**
Number of Queries	43	200
Average Length of Queries	32.5 characters	47
Query source	Bing questions	Yahoo! Answers
Sample Query	“hydrogen is a liquid below what temperature”	“why doesn t glue stick to the inside of the bottle“

The query sets utilised for evaluation are the train sets from both datasets. Although all query variations are able to generate multiple query variations per original query, given cost constraints and the degree of labour required in manually annotating and verifying the query variations, each query variation was performed only once per query.

4.1.1 LLM Query Variation

Given the original query q_1 a matching query variation q_0 is generated using the following prompt in the OpenAI GPT-4 API interface:

> “Paraphrase the following query and provide a query that conveys the same meaning. The query must be grammatically correct and clearly written:”

Figure 8 shows the prompts provided to the GPT API and their use to generate a useful query variation. The prompt provided to GPT was experimented with substantially to tailor query variations to be relevant and appropriate to the context. Subtle differences such as modifying the prompt to ‘Rephrase the following…’ or ‘…maintains query semantics…’ still result in sensical responses, but without query phrasing that is sufficiently natural.

Experimentation with the GPT Instantiation Context additionally flavours the returned response. The context was trialled as ‘You are an interested internet user’, ‘You are a curious human with an information need’, ‘You are a human’ and a host of other contexts. However, for reproducibility and maintaining the construction of GPT-4 as

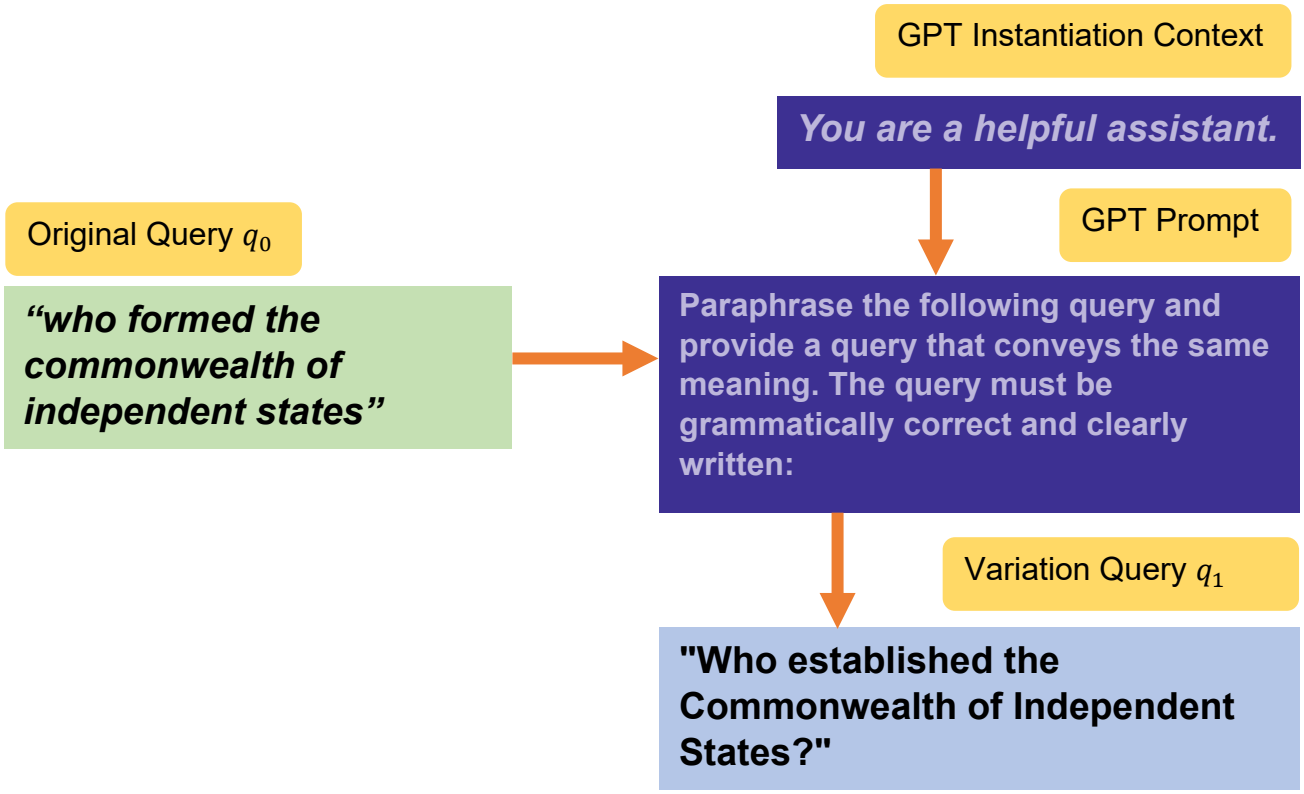


Figure 8; Process flow for generating a query variation using OpenAI GPT API

used by ChatGPT – where users cannot edit the model context, the instantiation context was left in a default state.

A python script was created (called ‘regenerate_query_open.py’ – can be viewed in Appendix B1) to access the GPT API and for each labelled query in a provided .csv file populated the returned data struct.

Each query was then manually inspected for maintaining reasonable meaning of the original. The GPT generated query variations were all comprehensively reflective of the original query, with in some cases significant spell correction and proper noun recognition taking place. Of significant importance is the level to which the generated

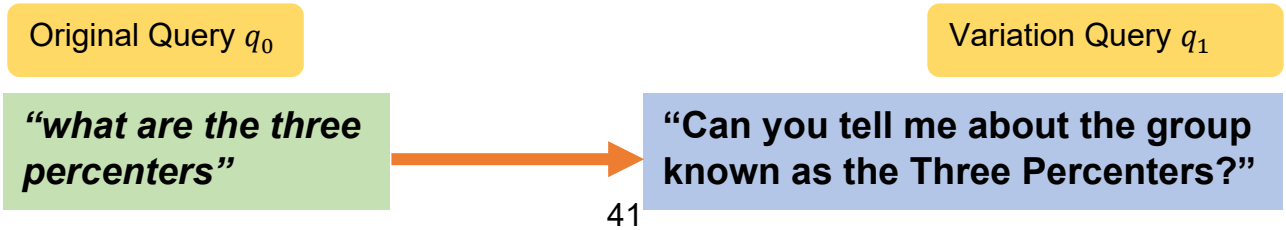


Figure 9; Example of increased specificity and query length in returned query variation

queries q_1 are more specific than their original counterparts. This can be seen in the original and variation query in Figure 9.
All queries generated by the method were deemed valid and reflective of the original queries' meaning that further evaluation with different ranking models was warranted.

4.1.2 Character Deletion

Given the original query q_1 a matching query variation q_0 is generated using a constructed program 'char_del.py'. The variation generator generates a random number (within the bounds of the length of original query) and deletes the corresponding character, without respect of character significance. Figure 10 shows an implementation of the function on a query sampled from TREC-DL-2019.

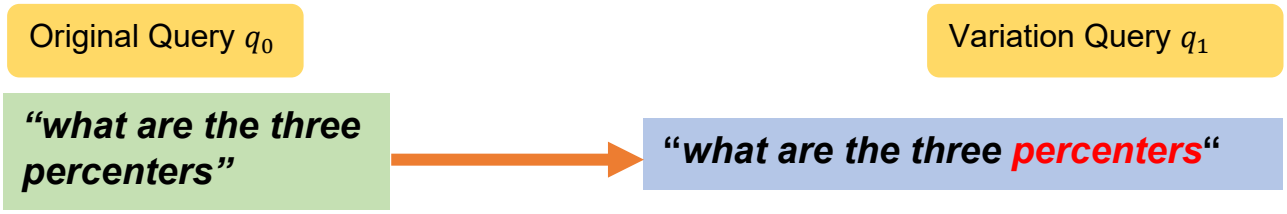


Figure 10; Demonstration of random character deletion on an original query (from TREC-DL-2019)

Stop words (i.e. insignificant words – 'the, is, at, which, and on') were not precluded from deletion, nor spaces. This aims to reflect typographical errors where a keypress was not initiated and to a lesser extent, user spelling errors.
All queries generated with this method (~250) were manually inspected for appropriate reflection of the intended focus. All queries were accepted and used for further evaluation, though queries in some cases were reasonably degraded in semantic meaning.

4.1.3 Lemmatization Query Variation

Given the original query q_1 a matching query variation q_0 is generated using a constructed program 'import_csv_for_lem.py'. The variation generator leverages the standard NLTK library implementation of WordNet Lemmatizer, which is easily imported and directed at standard data files. The implementation does not attempt to lemmatize stop words, but does lemmatize every word present within the original query, returning all lemmas in the variation query (q_1).

The lemma chosen is informed by positioning within the text, the term itself and therefore the associated term key values (adjective, noun, etc.).

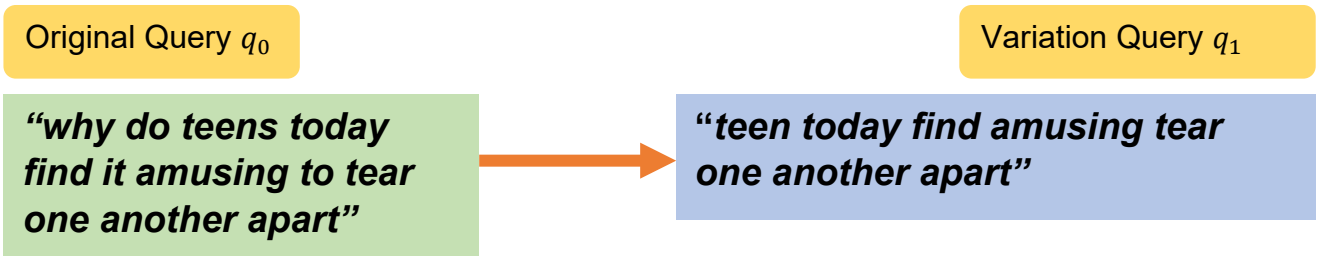


Figure 11; Demonstration of lemmatisation on an original query from ANTIQUE

Significant degradation was present in the returned query generations, as some context was stripped and stop words were removed. Additionally, no spell-checking was performed so queries were largely utilised as shorter trunked versions of themselves. As seen in Figure 11, the lemmatised sentence, removes the contextual ‘task’ term “why” at the beginning of the query, marking the query more akin to a news article or statement than a query. However, lemma sentences are reflective of users trying to reflect key search terms and reduce the spread of their search and thus should be considered.

4.2 Ranking Models

This study employs a range of ranking algorithms, extending from conventional lexical techniques, exemplified by BM25, to advanced neural ranking mechanisms, inclusive of Kernelized Neural Ranking Models (KNRM) and those integrating transformer-based linguistic models such as Bidirectional Encoder Representations from Transformers (BERT). In all experimental scenarios, an initial retrieval is conducted using BM25, subsequently re-ranking the top 100 results utilizing neural rankers, a method validated by prior research [19]

The BM25 [53] model and the Relevance Model 3 (RM3) [85] are utilized according to standard hyperparameters and implementations as provided by the PyTerrier framework. Kernel-based neural ranking models, specifically KNRM [75] and Convolutional KNRM (CKNRM) [91], are trained employing the OpenNIR platform defaults on datasets from TREC-DL-2019 and ANTIQUE [114].

For transformer-based ranking methods, EPIC [76] and BERT [24] (also recognized as monoBERT) undergo fine-tuning on the bert-base-uncased architecture. EPIC processes queries and documents independently, while monoBERT combines them,

leveraging the [CLS] token representation for prediction. The T5 model utilizes the monoT5 adaptation from the PyTerrier T5 plugin, incorporating pre-trained MSMarco weights as released by the original monoT5 authors [26].

4.2 Evaluation

All evaluation was performed using the Google Colaboratory computing service which is self-described as:

> "Colab is a hosted Jupyter Notebook service that requires no setup to use and provides free access to computing resources, including GPUs and TPUs. Colab is especially well suited to machine learning, data science, and education."
>
> - Google Colaboratory

The code for all evaluation is available in the author's GitHub repository which links to the implemented Colab notebook (https://github.com/Pikilman/thesis). Prior implementation of similar methods by Penha, G et al. have been extended and duplicated in some areas, to replicate and provide further context to established query robustness/variation efforts. Scripts authored by Penha, G et al. have been imported to the Colab Notebook or otherwise implemented directly. The code also leans heavily on several imported libraries and tools, most notably PyTerrier, TensorFlow, NLTK and a host of other helpful requirements. These are mentioned in further detail in the 'requirements.txt' document linked in Colab.

Measurement and comparison of model retrieval success was implemented by comparing NDCG@10 scores, though other scores are also generated.

OHS Risk and Ethic Assessment

Risk. This work is conducted in the low-risk laboratory covered by general OHS laboratory rules. Basic electrical safety and care are required but no specialised training.

Ethic. The datasets used in this project are all publicly available and for academic purposes. There is no personally identifiable information or privacy concerns with the data to be used.

5 Results

5.1 Query Generation

5.1.1 GPT Query Generation

The query variations generated by the LLM OpenAI GPT method are on average 50% longer than the original query Figure 12

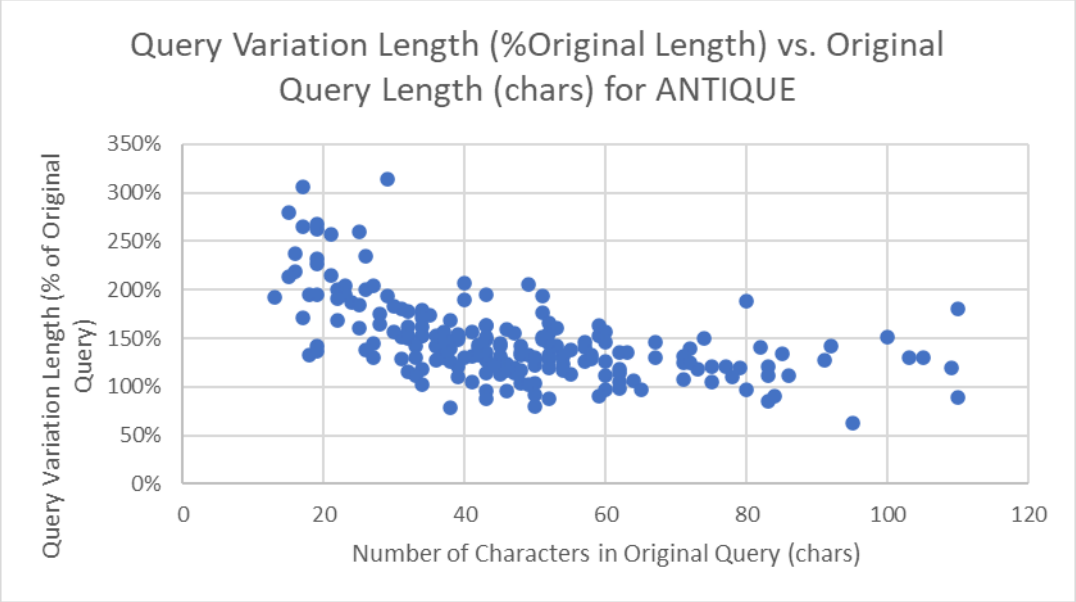


Figure 12; Increase in query variation length as a function of original query length (reported as a percentage). Shorter queries are more often represented by longer generated queries. For ANTIQUE dataset

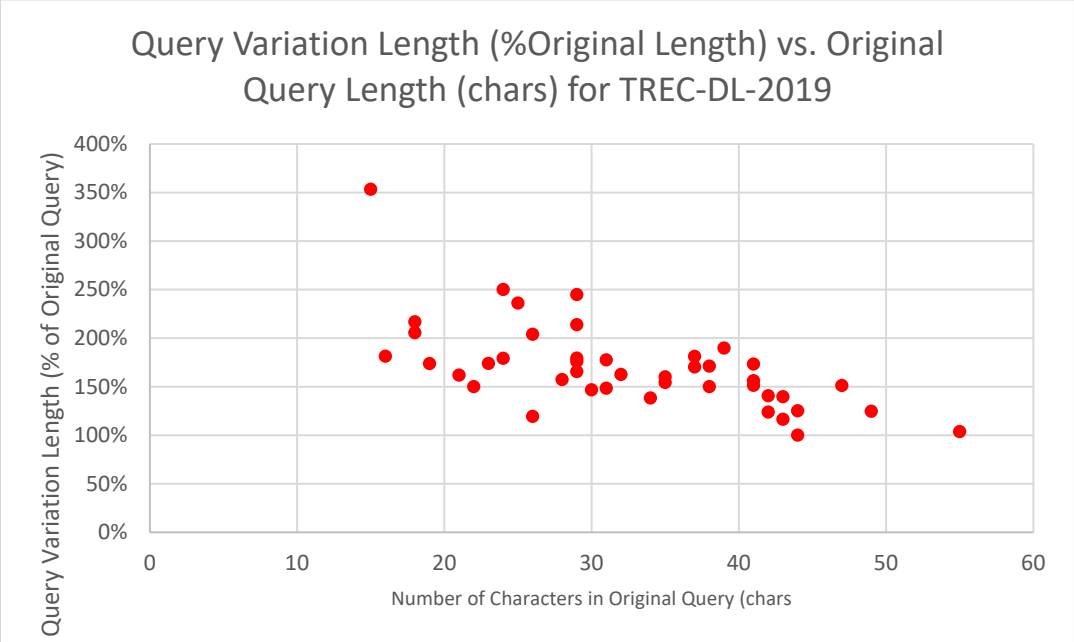


Figure 13; Increase in query variation length as a function of original query length (reported as a percentage). Shorter queries are more often represented by longer generated queries. For TREC-DL-2019 dataset

It is unclear why query generation with GPT-4 necessarily increases the length of the query. Token sizes are considerably smaller than 20 chars, but it is worth noting the minimum query variation lengths for both datasets shown in Table 1. No queries generated by the LLM GPT method were under 24 characters in length.

Table 1; Comparison between different query sets and their lengths after variation

Query Dataset	ANTIQUE	ANTIQUE Variations	% of Original Query	TREC-DL-2019	TREC-DL-2019 Variations	% of Original Query
Max Query Length (chars)	110	199	314%	55	74	353%
Min Query Length (chars)	13	24	63%	15	29	100%

Short, typically ‘How’ questions were most likely to be lengthened considerably. These changes could be due to hard-coded minimum generation limits, a goal by the LLM to remain conversational (as in implementation within ChatGPT) and overfill query gaps or due to qualification by the LLM in the presence of controversial or explicit questions.

An example of qualifying potentially controversial queries, and the specificity and length of the returned query as consequence, is given in the ANTIQUE dataset as follows:
“what s the best asian and why” -> (becomes) -> “Could you clarify which is the finest Asian cuisine and provide reasons for its prominence?”

Presumably to avoid racist undertones, the returned query is overly padded with specific and more sophisticated language, as well as more conversationally based text e.g. “Could **you**”.
Additionally, the queries generated are occassionally more specific than the original query, which though it represents a greater comprehension of the query, might not reflect the query of a user seeking information – e.g. the LLM might not be curious

enough and potentially prefills its query with knowledge of the answer. Given that query variations are only generated once per original query, significant specificity in the variation might preclude more general and relevant documents. Hence, the fusion of multiple query variations might be more comprehensive in communicating the information need of a user.

5.1.2 Character Deletion Query Variation

All queries generated by the character deletion method were as expected. A random selection for which character is removed results in, unsurprisingly, an equivalent number of queries all of which are one character length shorter. The method was fast and easily run on a large set of queries like ANTIQUE, utilising no extensive libraries or calculation. On average, queries lost 2.56% of their total character length, with 2.3% of all spaces within the query set being removed.

5.1.3 Lemmatisation Query Variation

The query variations generated by the NLTK lemmatisation method are on average 26.4% and 43% shorter than the original queries in TREC-DL-2019 and ANTIQUE respectively. Figure 14 and Figure 15 show for both datasets a consistent decrease in query length for the variation method.

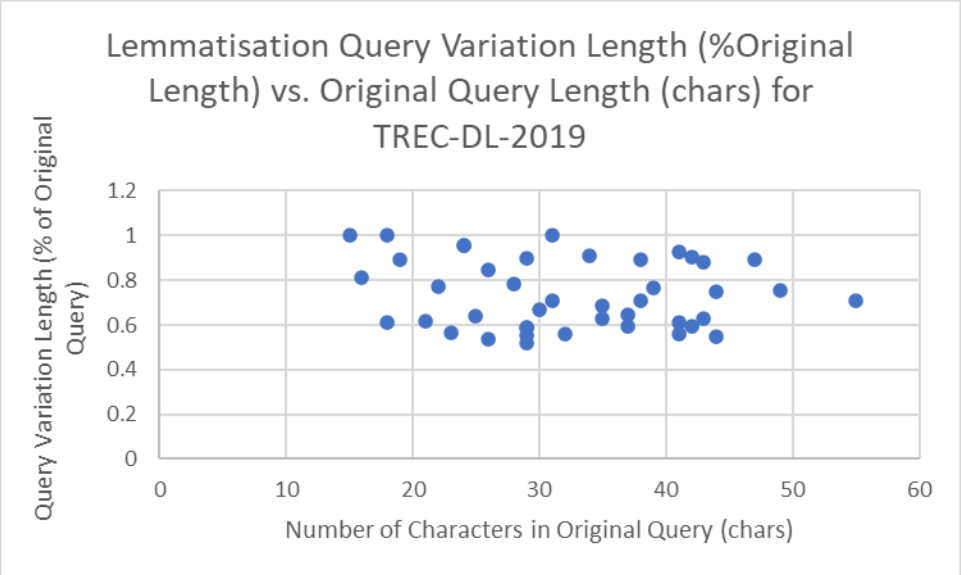


Figure 14; Increase in query variation length as a function of original query length (reported as a percentage). Perturbed queries are all either equal or shorter in length to original queries. For TREC-DL-2019 dataset

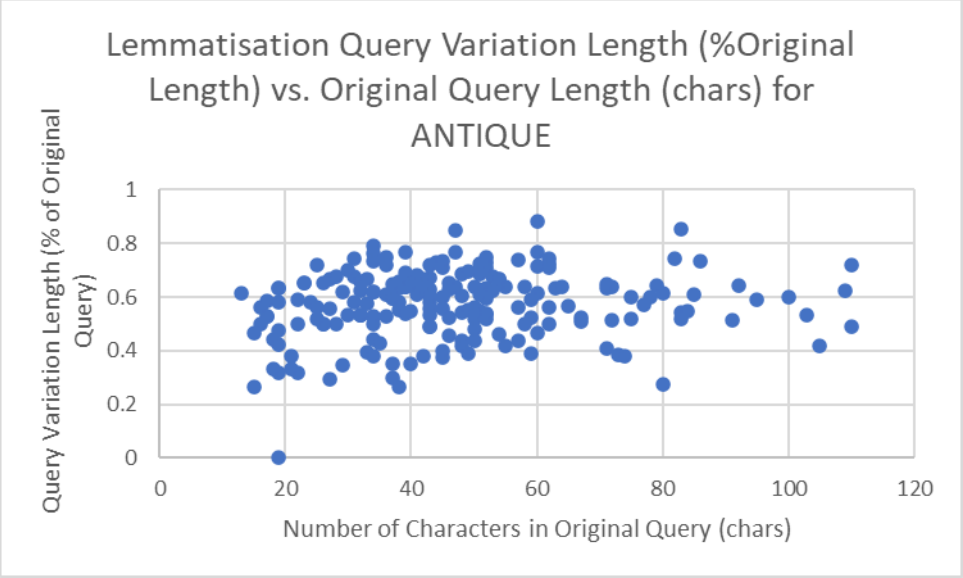


Figure 15; Perturbed queries are all either equal or shorter in length to original queries. For ANTIQUE dataset

Of particular note is the query in ANTIQUE which was reduced to a length of 0 characters, from a healthy 19. The query in question is as follows:

"how is it being you"

For contrast, the LLM generative method considers "What is it like being you?" as a valid transformation of the original query.

This is an exaggerated but plausible outcome of using lemmatisation whereby stop words are ignored and only lemmas of valid non-stop word terms are output. 'Insignificant' filler text that conveys subtle meanings and information needs is removed in favour of words with notable roots. This is largely due to the heavy-handed implementation of lemmatization within the variation generator, which does not attempt to maintain stop words (skipping over them in the lemmatization step) and instead opts to not join them to the output query.

Thus semantic meaning is often lost especially through removal of stop words like "how" which provides context to the original query. This effect is exacerbated by preprocessing stemming which could further distort the transformed query.

5.2 Model Effectiveness

To assess the stability of our trifurcated ranking architectures—conventional, neural network-driven, and transformer-infused—the study instituted a comparative analysis on performance metrics after the substitution of canonical queries with their modified

counterparts. Exhibited in Table 2 are the experimental outcomes pertinent to the TREC-DL-2019 and ANTIQUE data corpuses. The tabulation delineates, on a per-row basis, the efficacy of each ranking framework (as explicated in the columnar annotations) in conjunction with the queries derived via each automated query modification technique.

Table 2; Comparative Efficacy (nDCG@10) Across Varied Retrieval Methodologies in TREC-DL-2019 and ANTIQUE Datasets Subject to Query Modifications. Emboldened figures represent peak values for respective models, blue text indicates highest peak values for query variation evaluations. Cell colours are used to delineate original query evaluation vs perturbated queries (green = original, blue = perturbed). All query variation methods result in statistically significant (P<0.05) effectivity drops across both datasets (ANTIQUE and TREC-DL-2019.

TREC-DL-2019								
Category	Query Variation Method	BM25	RM3	KNRM	CKNRM	EPIC	BERT	T5
	Original Query	**0.4795**	**0.5156**	**0.5015**	**0.4931**	**0.6449**	**0.624**	**0.6997**
Lemmatisation	**'nltk_lemma'**	0.3644	0.3867	0.3611	0.3452	0.5159	0.5117	0.5598
Misspelling	**'del_space'**	0.2685	0.2990	0.3059	0.3057	0.3289	0.3058	0.3778
LLM Generation	**'regenerate_-query'**	0.3740	0.4022	0.4062	0.4043	0.5095	0.4930	0.6017
ANTIQUE								
Category	Query Variation Method	BM25	RM3	KNRM	CKNRM	EPIC	BERT	T5
	Original Query	**0.2286**	**0.2169**	**0.2175**	**0.2065**	**0.2663**	**0.4212**	**0.3338**
Lemmatisation	**'nltk_lemma'**	0.2109	0.1979	0.2069	0.2049	0.2449	0.3418	0.3010
Misspelling	**'del_space'**	0.1588	0.1556	0.1788	0.1889	0.2879	0.2824	0.2759
LLM Generation	**'regenerate_-query'**	0.1827	0.1735	0.1740	0.1652	0.2129	0.3367	0.2668

The evaluation metric used (NDCG@10) shows significant degradation in retrieval capability in the event of query variation, with an average effectiveness drop of

45.36% across both datasets (TREC-DL-2019 and ANTIQUE). No query variations evaluated across either data set was more effective in retrieval than the original query and by statistically significant margins ($P<0.05$).

This answers our main research question indicating that retrieval pipelines are not robust to query variations. Further, GPT-4 LLM based query transformation technique tested does not appreciably improve efficacy of any of the ranking pipelines utilised throughout the study. **Traditional, neural and pre-trained ranking models are not robust to natural language query variations.**

6 Discussion

The evaluation of different retrieval pipelines' efficacy for different query variation techniques shows that the tested ranking models are not robust to user query variations.

Lemmatization

The significant decrease in effectivity over lemmatized query variations may be attributed to removal of retrieval relevant stop words, especially on short queries or queries constructed of a majority of stop words. Significant improvements in NDCG@10 might be seen if further quality control was employed to ensure adequate query comprehension. Mitigation might also be achieved with longer query lengths or on a larger test set, such that one anomalous query does not constitute >2% of the test set (as was the case with a query being reduced to a zero character length query variation)

Although lemmatization does not simply remove suffixes (as is the approach of stemming), considering term context and converting to meaningful base form lemmas, key words with no supporting stop words fail to convey adequate context and information need.

Additionally, preprocessing – which typically incorporates a stemming process to strip and skeletonise search terms – when compounded with lemmatized terms might well dissolve the semantic meaning of any terms proffered to the retrieval system. For example, if all suffixes have already been removed, application of a second stemming step might incorrectly strip trailing characters in place of suffixes, corrupting query construction.

Character Deletion

Character deletion query variations had a marked effect in system retrieval, incurring a 50% decrease in retrieval effectiveness. Traditional and to some extent Neural (KNRM CKRNM) methods suffered the greatest drops in effectiveness. This may be attributed to the bag of words approach adopted by BM25 and similarly probabilistic frameworks for RM3. Additionally, more brittle neural network approaches like KNRM and CKNRM appear less robust to small perturbations in the original query. Large, pretrained language models like BERT, EPIC and T5 are less effected, exhibiting a slightly lower decrease in performance. This might be reflective of the training sensibilities of these models, where tokens are masked or span masked. Such a small perturbation as a single character being removed having such a marked effect indicates that retrieval systems are not robust to user query variations, as typographical errors such as missed keys are entirely reasonable and expected of practical use.

Large Language Modelling Query Generation

The queries generated by GPT-4 consistently longer than their original counterparts – only 13 out of ~250 queries were shorter than their original counterparts. This is reflective of the conversational, over-qualified and heightened specificity present in the retuned query variations. In particular, explicit original queries are subject to greater qualification and abstraction, most likely due to efforts to censor some actions on ChatGPT over safety and ethical concerns.

GPT-4 exhibits exceptional comprehension of search queries, such that it frings on being overly prescient and inciting and answer within the query. **GPT-4 is not curious enough**, at least in the instantiations used within this study. Although model context can be adjusted as well as response request prompts, further experimentation is required to implement greater efficacy.

GPT-4 excels at interpreting meaning from given search queries, occasionally to the detriment of nuanced and open questions. However, for recognising proper nouns and correcting spelling and grammar, GPT-4 is exceptional.

Utilising GPT-4 API calls to generate query variations was expensive ($3 for 400 calls), and calls to the API are rate limited, significantly reducing applicability of such a method being applied to practical queries. While this cost is expected to come

down as the state-of-the-art advances, costs are prohibitive to any large-scale application.

7 Conclusions and Future Work

Information retrieval pipelines that leverage effective neural ranking models need to be robust to user query variations. While studies have focused on perturbations to document passages and adversarial user queries, there exists a need to evaluate practical user query transitions with present neural models. Novel and established query transition methods were utilised to realise practical user variances. Methodologies were devised for three categories of query variations: character deletion, word lemmatization and GPT-4 generated queries, whilst maintaining query semantics. The quality of these generated queries was assessed, though no queries were removed from the query sets despite degradation, due to the goal of reflecting realistic user implementation (users do not rewrite their queries as the original). Empirical evaluation using two distinct datasets revealed a substantial diminution in retrieval efficacy, with an average decline of 45% when compared with the original queries from the test corpora.

The findings accentuate the necessity for the development of test collections encompassing a breadth of query variants to enhance the assessment of model effectiveness. Prospective research should address the creation of superior query variation generation systems that can modify query semantics without altering the underlying information need, as well as strategies to fortify the resilience of extant ranking mechanisms.

Future Work

Future work might focus on utilising GPT-4 as a query generator to augment a provided user query, to mitigate poorly directed overly specific queries, whilst gaining comprehensive insight as to the users' information need. Rank fusion and multi-query comprehension may benefit from the generative and infinitely callable capabilities of GPT-4.

Additionally, evaluation on larger query train sets would mitigate overrepresentation of poor-quality query variations.

Appendix A

B.1 ‘regenerate_query.py’

The following program is written in python, makes a call to the OpenAI GPT-4 API and populates a csv file. A private key was used to authorise calls to the API and has been redacted.

```
import openai
import csv

# Initialize the OpenAI API client
openai.api_key = REDACTED

def get_paraphrase(query):
    """Gets a paraphrased query using the ChatGPT-4 API."""
    response = openai.ChatCompletion.create(
        model="gpt-4",
        messages=[
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user", "content": f"Paraphrase the following query and provide a query that conveys the same meaning. The query must be grammatically correct and clearly written: {query}"}
        ]
    )
    return response.choices[0].message['content'].strip()

def main():
    # Input and output CSV filenames
    input_filename = r'C:\Users\olica\Downloads\unique_queries_antique.csv'
    output_filename = r'C:\Users\olica\Downloads\paraphrased_queries_antique.csv'

    # Lists to store read data and generated paraphrases
    queries_with_ids = []
```

```
    # Read input CSV
    with open(input_filename, 'r', newline='', encoding='utf-8') as csvfile:
        reader = csv.reader(csvfile)
        next(reader)  # skip header
        for row in reader:
            query_id, original_query = row
            queries_with_ids.append({'id': query_id, 'query': original_query})

    # CSV file setup for output
    with open(output_filename, 'w', newline='', encoding='utf-8') as csvfile:
        fieldnames = ['id', 'original_query', 'paraphrased_query']
        writer = csv.DictWriter(csvfile, fieldnames=fieldnames)

        # Write the header
        writer.writeheader()

        # Iterate over each query, get its paraphrase, and write to the CSV
        for item in queries_with_ids:
            paraphrased = get_paraphrase(item['query'])
            writer.writerow({'id': item['id'], 'original_query': item['query'],
'paraphrased_query': paraphrased})

if __name__ == '__main__':
    main()
```

Bibliography

1. Bailey, P., et al., *User Variability and IR System Evaluation*, in *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2015, Association for Computing Machinery: Santiago, Chile. p. 625–634.
2. Bailey, P., et al., *UQV100: A Test Collection with Query Variability*, in *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*. 2016, Association for Computing Machinery: Pisa, Italy. p. 725–728.
3. Buckley, C. and J.A. Walz. *The TREC-8 Query Track*. in *Text Retrieval Conference*. 1999.
4. Zuccon, G., J. Palotti, and A. Hanbury, *Query Variations and their Effect on Comparing Information Retrieval Systems*, in *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*. 2016, Association for Computing Machinery: Indianapolis, Indiana, USA. p. 691–700.
5. Moffat, A., et al., *Pooled Evaluation Over Query Variations: Users are as Diverse as Systems*, in *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*. 2015, Association for Computing Machinery: Melbourne, Australia. p. 1759–1762.
6. Benham, R. and J.S. Culpepper, *Risk-Reward Trade-offs in Rank Fusion*, in *Proceedings of the 22nd Australasian Document Computing Symposium*. 2017, Association for Computing Machinery: Brisbane, QLD, Australia. p. Article 1.
7. Belkin, N.J., et al., *Combining the evidence of multiple query representations for information retrieval.* Information Processing & Management, 1995. **31**(3): p. 431-448.
8. Benham, R., et al., *Boosting Search Performance Using Query Variations.* ACM Transactions on Information Systems, 2019. **37**(4): p. 1-25.

9. Lu, X., et al., *Relevance Modeling with Multiple Query Variations*, in *Proceedings of the 2019 ACM SIGIR International Conference on Theory of Information Retrieval*. 2019, Association for Computing Machinery: Santa Clara, CA, USA. p. 27–34.
10. Zendel, O., et al. *Information Needs, Queries, and Query Performance Prediction*. ACM.
11. Benham, R., et al. *Towards Efficient and Effective Query Variant Generation*. 2018. Germany: Rheinisch-Westfaelische Technische Hochschule Aachen * Lehrstuhl Informatik V.
12. Chakraborty, A., D. Ganguly, and O. Conlan. *Retrievability based Document Selection for Relevance Feedback with Automatically Generated Query Variants*. ACM.
13. Penha, G., A. Câmara, and C. Hauff. *Evaluating the Robustness of Retrieval Pipelines with Query Variation Generators*. in *European Conference on Information Retrieval*. 2021.
14. MacAvaney, S., et al., *ABNIRML: Analyzing the Behavior of Neural IR Models.* arXiv pre-print server, 2020.
15. Hashemi, H., et al., *ANTIQUE: A Non-Factoid Question Answering Benchmark.* arXiv pre-print server, 2019.
16. Bajaj, P., et al., *MS MARCO: A Human Generated MAchine Reading COmprehension Dataset.* arXiv pre-print server, 2018.
17. Devlin, J., et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019, Minneapolis, Minnesota: Association for Computational Linguistics. 4171-4186.
18. Raffel, C., et al., *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.* arXiv pre-print server, 2020.
19. Lin, J., R. Nogueira, and A. Yates, *Pretrained Transformers for Text Ranking: BERT and Beyond.* arXiv pre-print server, 2020.

20. Câmara, A. and C. Hauff, *Diagnosing BERT with Retrieval Heuristics*. 2020, Springer International Publishing. p. 605-618.
21. Padigela, H., H. Zamani, and W, *Investigating the Successes and Failures of BERT for Passage Re-Ranking.* arXiv pre-print server, 2019.
22. Rennings, D., F. Moraes, and C. Hauff, *An Axiomatic Approach to Diagnosing Neural IR Models*. 2019, Springer International Publishing. p. 489-503.
23. Wu, C., et al., *Are Neural Ranking Models Robust?* arXiv pre-print server, 2022.
24. Nogueira, R. and K. Cho, *Passage Re-ranking with BERT.* arXiv pre-print server, 2020.
25. Penha, G. and C. Hauff, *Curriculum Learning Strategies for IR*. 2020, Springer International Publishing. p. 699-713.
26. Nogueira, R., Z. Jiang, and J. Lin, *Document Ranking with a Pretrained Sequence-to-Sequence Model.* arXiv pre-print server, 2020.
27. Clark, K., et al., *ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators.* arXiv pre-print server, 2020.
28. Yang, W., H. Zhang, and J. Lin, *Simple Applications of BERT for Ad Hoc Document Retrieval.* arXiv pre-print server, 2019.
29. Liu, J., et al., *Order-Disorder: Imitation Adversarial Attacks for Black-box Neural Ranking Models.* arXiv pre-print server, 2023.
30. Raval, N. and M. Verma, *One word at a time: adversarial attacks on retrieval models.* arXiv pre-print server, 2020.
31. Bailey, P., et al. *Retrieval Consistency in the Presence of Query Variations*. ACM.
32. Belkin, N., et al. *The effect multiple query representations on information retrieval system performance*. 1993. ACM.
33. Vogt, C.C. and G.W. Cottrell, *Fusion Via a Linear Combination of Scores.* Information Retrieval, 1999. **1**(3): p. 151-173.

34. Gustavo Penhal, A.C., Claudia Hauff, *Evaluating the Robustness of Retrieval Pipelines with Query Variation Generators.* CoRR, 2022. **abs/2111.13057**.
35. Craswell, N., et al., *Overview of the TREC 2021 deep learning track.* 2022.
36. Nogueira, R., et al., *Document Expansion by Query Prediction.* arXiv pre-print server, 2019.
37. Nogueira, R. and J. Lin, *From doc2query to docTTTTTquery An MS MARCO passage retrieval task 1 μpublication.* 2019.
38. Kailash and H. Proenca, *Information Retrieval: Recent Advances and Beyond.* arXiv pre-print server, 2023.
39. Karpukhin, V., et al., *Dense Passage Retrieval for Open-Domain Question Answering.* arXiv pre-print server, 2020.
40. Dwivedi, S.K. and V. Singh, *Research and Reviews in Question Answering System.* Procedia Technology, 2013. **10**: p. 417-424.
41. Ji, Z., Z. Lu, and H. Li, *An Information Retrieval Approach to Short Text Conversation.* arXiv pre-print server, 2014.
42. Chen, H., et al., *A Survey on Dialogue Systems.* ACM SIGKDD Explorations Newsletter, 2017. **19**(2): p. 25-35.
43. Gillick, D., et al. *Learning Dense Representations for Entity Retrieval*. 2019. Hong Kong, China: Association for Computational Linguistics.
44. Zheng, Z., et al., *BERT-QE: Contextualized Query Expansion for Document Re-ranking.* arXiv pre-print server, 2020.
45. Mitra, B., F. Diaz, and N. Craswell. *Learning to Match using Local and Distributed Representations of Text for Web Search*. International World Wide Web Conferences Steering Committee.
46. Salton, G., A. Wong, and C.S. Yang, *A vector space model for automatic indexing.* 1975. **18**(11 %J Commun. ACM): p. 613–620.
47. Croft, W.B., D. Metzler, and T. Strohman. *Search Engines - Information Retrieval in Practice*. 2009.

48. Furnas, G.W., et al., *The vocabulary problem in human-system communication.* 1987. **30**(11 %J Commun. ACM): p. 964–971.
49. Zhang, Z. and P. Zweigenbaum, *GNEG: Graph-Based Negative Sampling for word2vec*, in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 2018, Association for Computational Linguistics. p. 566-571.
50. Li, H. and J. Xu, *Semantic Matching in Search.* Foundations and Trends® in Information Retrieval, 2014. **7**(5): p. 343-469.
51. Salton, G., *Some experiments in the generation of word and document associations*, in *Proceedings of the December 4-6, 1962, fall joint computer conference*. 1962, Association for Computing Machinery: Philadelphia, Pennsylvania. p. 234–250.
52. Aizawa, A., *An information-theoretic perspective of tf–idf measures.* Information Processing & Management, 2003. **39**(1): p. 45-65.
53. Robertson, S. and H. Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in Information Retrieval, 2009. **3**: p. 333-389.
54. Zhai, C., *Statistical Language Models for Information Retrieval*. Vol. 1. 2008.
55. Jurafsky, D. and J.H. Martin, *Speech and Language Processing (2Nd Edition)*. 2009, Upper Saddle River, NJ, USA: Prentice-Hall, Inc.
56. Liu, T.-Y., *Learning to Rank for Information Retrieval*. Vol. 3. 2010. 904.
57. Li, H., *Theory of Learning to Rank*, in *Learning to Rank for Information Retrieval and Natural Language Processing*, H. Li, Editor. 2015, Springer International Publishing: Cham. p. 81-86.
58. Huang, P.-S., et al., *Learning Deep Structured Semantic Models for Web Search using Clickthrough Data.* 2013.
59. Mitra, B. and N. Craswell, *Neural Models for Information Retrieval.* arXiv pre-print server, 2017.

60. Guo, J., et al. *A Deep Relevance Matching Model for Ad-hoc Retrieval*. ACM.
61. Guo, J., et al., *A Deep Look into Neural Ranking Models for Information Retrieval.* arXiv pre-print server, 2019.
62. Vaswani, A., et al., *Attention Is All You Need.* arXiv pre-print server, 2017.
63. Craswell, N., et al., *Overview of the TREC 2020 deep learning track.* arXiv pre-print server, 2021.
64. Kwiatkowski, T., et al., *Natural Questions: A Benchmark for Question Answering Research.* Transactions of the Association for Computational Linguistics, 2019. **7**: p. 453-466.
65. Guo, J., et al., *Semantic Models for the First-Stage Retrieval: A Comprehensive Review.* ACM Transactions on Information Systems, 2022. **40**(4): p. 1-42.
66. Chen, R.-C., et al., *Efficient Cost-Aware Cascade Ranking in Multi-Stage Retrieval.* Sigir '17, 2017: p. 445–454.
67. Matveeva, I., et al., *High Accuracy Retrieval with Multiple Nested Ranker.* Sigir '06, 2006: p. 437–444.
68. Liu, S., et al. *Cascade Ranking for Operational E-commerce Search*. ACM.
69. Dietz, L., et al., *TREC Complex Answer Retrieval Overview*, in *Proceedings of The Twenty-Sixth Text REtrieval Conference, TREC 2017, Gaithersburg, Maryland, USA, November 15-17, 2017*, E.M. Voorhees and A. Ellis, Editors. 2017, National Institute of Standards and Technology (NIST).
70. Nguyen, T., et al., *MS MARCO: A Human Generated MAchine Reading COmprehension Dataset*. 2016.
71. Nogueira, R., et al., *Multi-Stage Document Ranking with BERT.* arXiv pre-print server, 2019.
72. Robertson, S.E. and S. Walker, *Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval*. 1994, Springer London. p. 232-241.
73. Robertson, S., et al., *Okapi at TREC-3.* 1995: p. 109-126.

74. Abdul-Jaleel, N.A., James, et al., *"UMass at TREC 2004: Novelty and HARD".* Computer Science Department Faculty Publication Series. 189., 2004.
75. Xiong, C., et al. *End-to-End Neural Ad-hoc Ranking with Kernel Pooling*. ACM.
76. Macavaney, S., et al. *Expansion via Prediction of Importance with Contextualization*. ACM.
77. Salton, G. and C. Buckley, *Term-weighting approaches in automatic text retrieval.* 1988. **24**(5 %J Inf. Process. Manage.): p. 513–523.
78. Salton, G., *Developments in Automatic Text Retrieval.* Science, 1991. **253**(5023): p. 974-980.
79. Robertson, S. and S. Walker, *Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval*. 1994. 232-241.
80. Amati, G., *BM25*, in *Encyclopedia of Database Systems*, L. Liu and M.T. ÖZsu, Editors. 2009, Springer US: Boston, MA. p. 257-260.
81. Harter, S.P., *A probabilistic approach to automatic keyword indexing. Part I. On the Distribution of Specialty Words in a Technical Literature.* 1975. **26**(4): p. 197-206.
82. Robertson, S.E. and K.S. Jones, *Relevance weighting of search terms.* 1976. **27**(3): p. 129-146.
83. Lv, Y. and C. Zhai, *When documents are very long, BM25 fails!*, in *Proceedings of the 34th international ACM SIGIR conference on Research and development in Information Retrieval*. 2011, Association for Computing Machinery: Beijing, China. p. 1103–1104.
84. Lavrenko, V., *A generative theory of relevance*. 2004, University of Massachusetts Amherst: United States -- Massachusetts. p. 216.
85. Lavrenko, V. and W.B. Croft, *Relevance based language models*, in *Proceedings of the 24th annual international ACM SIGIR conference on Research and development in information retrieval*. 2001, Association for Computing Machinery: New Orleans, Louisiana, USA. p. 120–127.

86. Zhai, C. and J. Lafferty, *A study of smoothing methods for language models applied to information retrieval.* 2004. **22**(2 %J ACM Trans. Inf. Syst.): p. 179–214.
87. Lv, Y. and C. Zhai, *A comparative study of methods for estimating query language models with pseudo feedback*, in *Proceedings of the 18th ACM conference on Information and knowledge management*. 2009, Association for Computing Machinery: Hong Kong, China. p. 1895–1898.
88. Carpineto, C. and G. Romano, *A Survey of Automatic Query Expansion in Information Retrieval.* 2012. **44**(1 %J ACM Comput. Surv.): p. Article 1.
89. Li, H., *A Short Introduction to Learning to Rank.* IEICE Transactions on Information and Systems, 2011. **E94.D**(10): p. 1854-1862.
90. Zamani, H., et al., *From Neural Re-Ranking to Neural Ranking: Learning a Sparse Representation for Inverted Indexing*, in *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*. 2018, Association for Computing Machinery: Torino, Italy. p. 497–506.
91. Dai, Z., et al., *Convolutional Neural Networks for Soft-Matching N-Grams in Ad-hoc Search*, in *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*. 2018, Association for Computing Machinery: Marina Del Rey, CA, USA. p. 126–134.
92. Hofst\"atter, S., et al., *Efficiently Teaching an Effective Dense Retriever with Balanced Topic Aware Sampling.* arXiv pre-print server, 2021.
93. MacAvaney, S., A. Cohan, and N. Goharian, *SLEDGE-Z: A Zero-Shot Baseline for COVID-19 Literature Search*. 2020.
94. Hui, K., et al., *PACRR: A Position-Aware Neural IR Model for Relevance Matching.* arXiv pre-print server, 2017.
95. Onal, K.D., et al., *Neural information retrieval: at the end of the early years.* 2017. **21**: p. 111 - 182.

96. Mitra, B. and N. Craswell, *An Introduction to Neural Information Retrieval t.* Foundations and Trends® in Information Retrieval, 2018. **13**(1): p. 1-126.
97. Tonellotto, N., *Lecture Notes on Neural Information Retrieval.* 2022. **abs/2207.13443**.
98. Liu, Y., et al., *RoBERTa: A Robustly Optimized BERT Pretraining Approach.* 2019. **abs/1907.11692**.
99. Sanh, V., et al., *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.* 2019. **abs/1910.01108**.
100. Lewis, M., et al. *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. in *Annual Meeting of the Association for Computational Linguistics*. 2019.
101. Fan, Y., et al., *Pre-training Methods in Information Retrieval.* 2021. **abs/2111.13853**.
102. Clark, K., et al., *ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators.* 2020.
103. Formal, T., et al., *SPLADE v2: Sparse Lexical and Expansion Model for Information Retrieval.* arXiv pre-print server, 2021.
104. Hashemi, H., et al. *ANTIQUE: A non-factoid question answering benchmark*. in *Advances in Information Retrieval: 42nd European Conference on IR Research, ECIR 2020, Lisbon, Portugal, April 14–17, 2020, Proceedings, Part II 42*. 2020. Springer.
105. Hofstätter, S. and A. Hanbury. *Let's measure run time! Extending the IR replicability infrastructure to include performance aspects*. in *OSIRRC@SIGIR*. 2019.
106. Nogueira, R., Z. Jiang, and J. Lin, *Document ranking with a pretrained sequence-to-sequence model.* 2020.
107. Jones, K.S., et al., *Report on the Need for and Provision of an 'ideal' Information Retrieval Test Collection*. 1975: University Computer Laboratory.
108. Moffat, A., et al. *Pooled Evaluation Over Query Variations*. ACM.

109. MacAvaney, S., et al., *Simplified Data Wrangling with ir_datasets.* 2021.
110. OpenAI, *GPT-4 Technical Report.* 2023. **abs/2303.08774**.
111. Ba, J.L., J.R. Kiros, and G.E. Hinton, *Layer normalization.* 2016.
112. Radford, A., et al. *Language Models are Unsupervised Multitask Learners*. 2019.
113. Brown, T., et al., *Language models are few-shot learners.* 2020. **33**: p. 1877-1901.
114. MacAvaney, S., *OpenNIR: A Complete Neural Ad-Hoc Ranking Pipeline*, in *Proceedings of the 13th International Conference on Web Search and Data Mining*. 2020, Association for Computing Machinery: Houston, TX, USA. p. 845–848.